



Data Parallelism Training on GPUs

Recap: ML on Hardware

- The CPU is not the only type of hardware that machine learning models are trained or run on.
- In fact, most modern DNN training happens not on CPUs but on GPUs.
- In this lecture, we'll look at why GPUs became dominant for machine learning training, and we'll explore what makes their architecture uniquely well-suited to large-scale numerical computation.

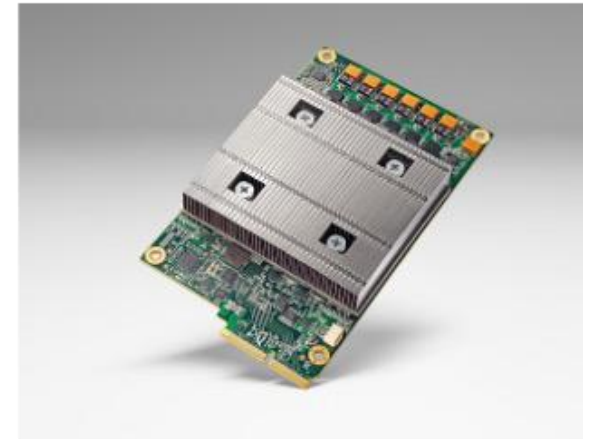


Modern ML Hardware

- CPUs
- GPUs



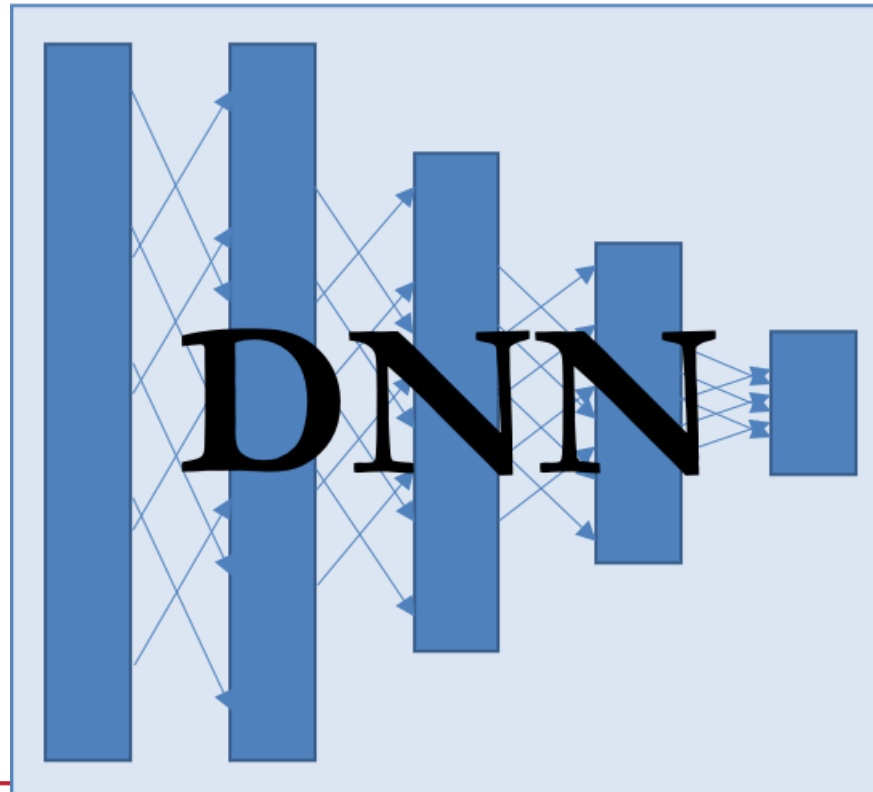
- FPGAs
- Specialized accelerators



Right now, **GPUs are dominant** for ML training...we'll get to why later

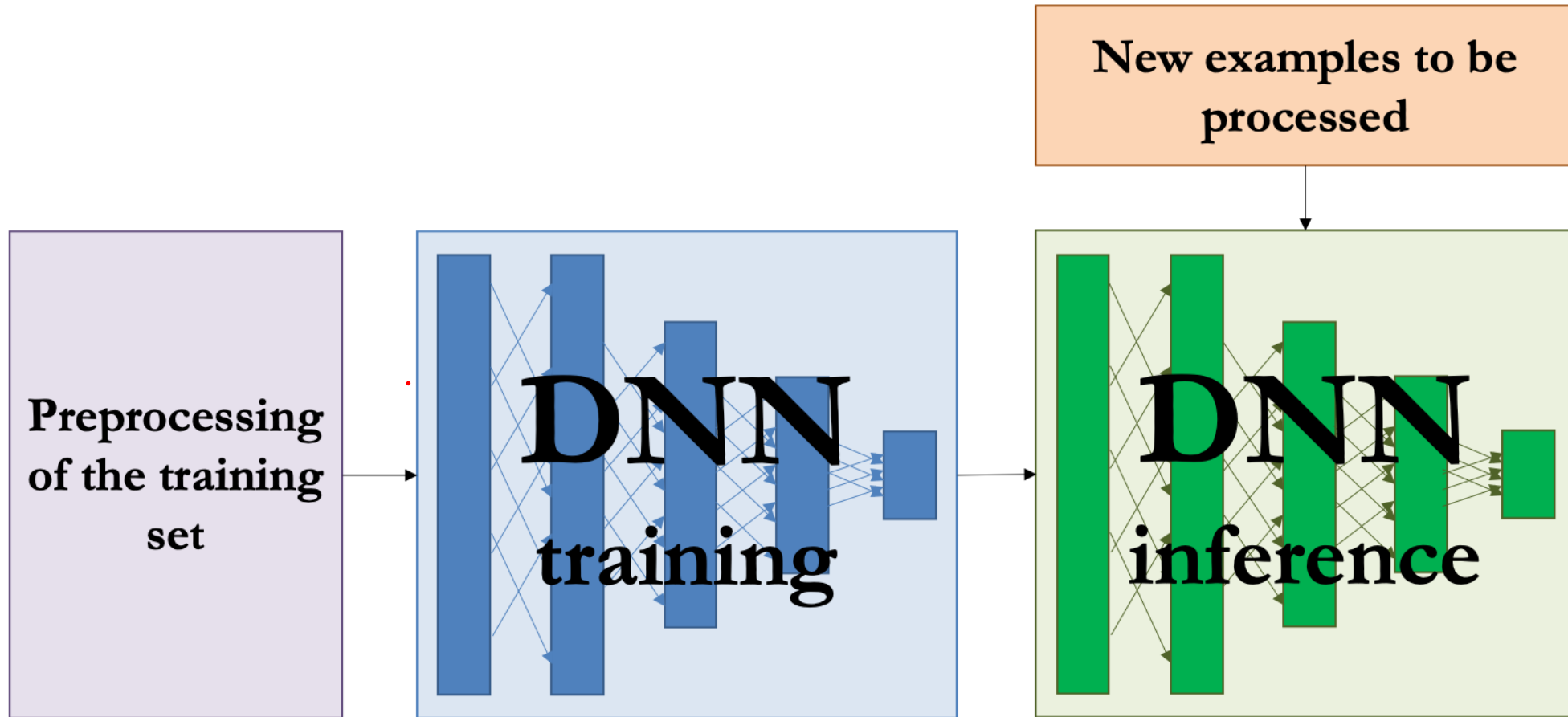
What does the modern ML pipeline look like?

- It's not just training a neural network.



What does the modern ML pipeline look like?

- The pipeline has many different components



Where can hardware help?

There's interest in hardware everywhere in the pipeline

- both adapting existing hardware architectures, and
- developing new ones

Active Learning Q: **What improvements can we get?**

- Lower latency inference
 - To be able to make real-time predictions
- Higher throughput training
 - To train on larger datasets to produce more accurate models
- Lower power cost
 - Especially important as data and model sizes scale

How can hardware help?

Speed up the basic building blocks of machine learning

- Major building block: matrix-matrix multiply
- Other major building blocks: convolution, attention

Add data/memory paths specialized to machine learning computations and workloads

- Example: having a local cache to store network weights
- Create application-specific functional units
- Not for general ML, but for a specific domain

Why are GPUs so popular for machine learning?



Why are GPUs so popular for training deep neural networks?

To answer this...we need to look at GPU architectures.



A brief history of GPU computing

- GPUs were originally designed to support the 3D graphics pipeline, driven by demand for videogames with ever-increasing graphical fidelity.



Important properties of 3d graphics rendering:

- Lots of opportunities for parallelism
 - rendering different pixels/objects in the scene can be done simultaneously
 - Lots of **numerical computations**
 - 3d rendering is based on geometry
 - Mostly polygons in space which are transformed, animated, and shaded
 - All of this is **numerical computation: mostly linear algebra**

GPU computing history (continued)

- The first era of GPUs ran a **fixed-function graphics pipeline**. They weren't programmed, but instead just configured to use a set of fixed functions designed for specific graphics tasks: mostly drawing and shading polygons in 3d space.
 - Even these GPUs were a little programmable via OpenGL calls, but you couldn't *program* them...in the sense of writing an ordinary imperative program that would run there



Programmable GPUs

- In the early 2000s, there was a shift towards
- **programmable GPUs.**
- Programmable GPUs allowed customization certain stages in the graphics pipeline by writing small programs called **shaders** which let developers process
- the vertices and pixels of the polygons they wanted to render in custom ways.
- And there was powerful economic incentive for hardware designers to increase the throughput of these GPU **shaders** so that game designers could render models with more polygons and higher resolution.

What did shaders need from hardware?

- These shaders needed to be **capable of very high- throughput parallel processing**
 - So that they could process and render very large numbers of polygons in a single frame of a 3d animation.
 - Intuition: if I am rendering multiple independent objects in a scene, I can process them in parallel.
- The shaders also needed **high throughput to high latency texture memory**
 - Texture memory accesses are essentially random
 - Needed lots of parallel contexts to be able to hide that latency

- The GPU supported parallel programs that were more parallel than those of the CPU.
- Unlike multicore CPUs, which supported computing different functions at the same time, the GPU computes the **same function on multiple elements of data**.
 - Example application: want to render a large number of triangles, each of which is lit by the same light sources.
 - Example application: want to transform all the objects in the scene based on the motion of the player. That is, the GPU could run the same function on a bunch of triangles in parallel, but couldn't easily compute a different function for each triangle.

- The GPU needed high bandwidth access to large texture memory to draw textures on models in real time.
 - This memory has **high latency**
- Unlike multicore CPUs, which have deep cache hierarchies, the GPU used parallelism to **hide latency**
 - Example application: when a shader reaches a lookup into texture memory, the GPU “core” will request the data; while the request is resolving, it runs other threads.
 - But actually switching to an independent program would be too expensive, so instead it does same program/different data

Two types of parallelism:

- **Data parallelism** involves the same operations being computed in parallel on many different data elements.
- **Task parallelism** involves different operations being computed in parallel
 - on either the same data or different data

Active Learning Question: **How are the types of parallelism we've discussed categorized according to this distinction?**

- SIMD/Vector parallelism?
- Multi-core/multi-thread parallelism?
- Distributed computing?

The general-purpose GPU

- Eventually, people started to use GPUs for tasks other than graphics rendering.
- However, **working within the structure of the graphics pipeline of the GPU placed limits on this.**
 - OpenGL is great for graphics, but annoying for ML
- To better support general-purpose GPU programming, in 2007 NVIDIA released **CUDA**, a parallel programming language/computing platform for general-purpose computation on the GPU.
 - Other companies (e.g. Intel/AMD) have competing products as well.
- Now, programmers no longer needed to use the 3d graphics API to access the parallel computing capabilities of the GPU.



A Revolution in GPU Computing

Several new applications:

- Training deep neural networks
- Cryptocurrencies

All based on running small highly parallel programs called “**kernels**” on the GPU.

- Not to be confused with the kernels of kernel learning!



An Illustration of CUDA

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}
```

This syntax launches N threads, each of which performs a single addition.

```
int main( )
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
    ...
}
```

It's straightforward to implement other ops

```
// Kernel definition
__global__ void VecMul(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] * B[i];
}

int main()
{
    ...
    // Kernel invocation with N threads
    VecMul<<<1, N>>>(A, B, C);
    ...
}
```

The general-purpose GPU: upsides

- Importantly, spinning up hundreds or thousands of threads like this could be reasonably fast on a GPU, while on a CPU this would be **way** too slow due to the overhead of creating new threads.
 - Because GPU threads run the same program, so they share resources for their program context
- Additionally, GPUs support many many more parallel threads running than a CPU.
 - Typical thread count for a GPU: tens of thousands.
 - Typical thread count for a CPU: at most a few dozen.

The general-purpose GPU: downsides

- Important downside of GPU threads of this type: they're data-parallel only.
 - You couldn't direct each of the individual threads to "do its own thing" or run its own independent computation
- Importantly: more **modern GPUs now support some level of task-parallelism.**
 - Threads are bound together into "warps" (of ~32 threads) that all (conditionally) execute the same instruction.
 - But threads in different warps have limited ability to run independently.

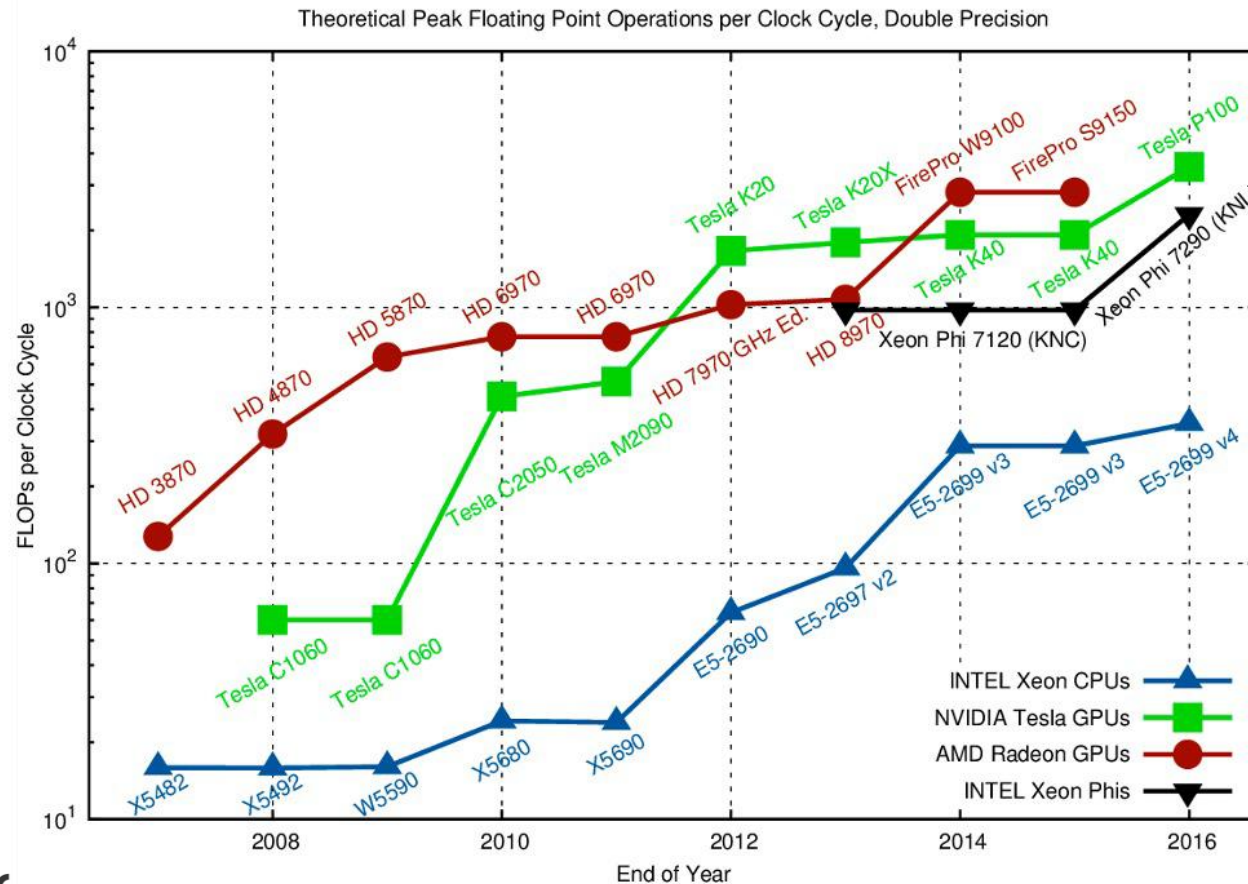
Summary: GPU vs CPU

- CPU is a general purpose processor
 - Modern CPUs spend most of their area on deep caches
 - This makes the CPU a great choice for applications with random or non-uniform memory accesses
- GPU is optimized for
 - more compute intensive workloads
 - streaming memory models

Machine learning workloads are compute intensive and often support streaming data flows.

FLOPS: GPU vs CPU

- FLOPS: floating point operations per second



**GPU FLOPS
consistently
exceed CPU
FLOPS**

From Karl Rupp's blog
<https://www.karlrupp.net/2016/08/flops-per-cycle-for-cpus-gpus-and-xeon-phis/>

This was the best diagram I could find that shows trends over time.

Memory Bandwidth: GPU vs CPU

- GPUs have higher memory bandwidths than CPUs
 - NVIDIA A100 80GB GPU has 2 TB/s memory bandwidth
 - Whereas an Intel Xeon Platinum CPU has only about 120 GB/s memory bandwidth at about the same time
- **But, this comparison is a bit unfair!**
 - GPU memory bandwidth is the **bandwidth to GPU memory**
 - e.g. on a PCIe2, bandwidth to system memory was only 32 GB/s for a GPU

- Active learning question: **Is deep learning more compute bound or more memory bound?**
 - i.e. is it more limited by the computational capabilities or memory capabilities of the hardware?

What limits deep learning?

- **Ideally: for MLPs it's compute bound**

- Why? Matrix-matrix multiply over $n \times n$ matrices takes $O(n^2)$ memory but $O(n^3)$ compute
- So as we scale wider, costs will be dominated by that compute

- **But often it is memory/communication bound**

- When we are doing convolutions with small filter sizes, compute becomes less dominant
- Especially when we have to load a lot of data
- Especially when we are running at large scale on a cluster
- Especially for transformers
- Especially when data/models do not fit in caches on the hardware

GPUs in machine learning

- Because of their high data parallelism, GPUs provide **an ideal substrate for large-scale numerical computation**.
 - In particular, GPUs can perform matrix multiplies very fast.
 - Just like BLAS on the CPU, there's an optimized library from NVIDIA "cuBLAS" that does matrix multiples efficiently on their GPUs.
 - There's even a specialized library of primitives designed for deep learning: cuDNN.
- Machine learning frameworks, such as TensorFlow and PyTorch, are designed to support computation on GPUs.
 - And training a deep net on a GPU can decrease training time by an order of magnitude.

How do ML frameworks support GPU computing?

A core feature of ML frameworks is GPU support. Approach:

- Represent the compute graph as linear-algebra operations.
- For each operation, call a hand-optimized kernel that computes that op on the GPU.
 - i.e. matrix-matrix multiply
 - i.e. convolution for a CNN
- When GPU code is not available for an op, run it on the CPU
 - but this happens rarely, only with uncommon architectures
- Stitch those ops together by calling them from Python
- and eventually make the results available back on the CPU

So should we always use GPUs?
Will we always use GPUs?

Maybe not!



Challengers to the GPU

- **More compute-intensive CPUs**
 - Like Intel's Phi line — promise same level of compute performance and better handling of sparsity
- **Low-power devices**
 - Like mobile-device-targeted chips
- **Configurable hardware like FPGAs and CGRAs**
- **Accelerators that speed up matrix-matrix multiply**
 - Like Google's Version 1 TPU
- **Accelerators special-designed to accelerate ML computations beyond just matrix-matrix multiply**
 - Version 2/3 TPUs
- Many competitors in this space



Effect of Hardware on Statistical Performance

- Sometimes, when we change device, we expect the same results: same learned model, same accuracy. **But this is not always the case. Why?**
- One reason: FP arithmetic is not associative, so reordering to get better performance on different hardware can change the results slightly!
 - This can even be the case for multiple runs on the same hardware.
- Another reason: randomness.
 - Different hardware platforms could use different pseudo-random generators or use random numbers in a different order.
- Another reason: hardware can do approximation to trade-off precision for hardware efficiency.
 - E.g. if the hardware runs on 16-bit floats, it can give different results.



Effect of Hardware on Statistical Performance

- Sometimes, when we change device, we expect the same results: same learned model, same accuracy. **But this is not always the case. Why?**
- **Generally, we expect specialized ML hardware to produce learned models that are similar in quality, but not necessarily exactly the same, as baseline chips (CPU/GPU).**

Programming ML Hardware

- One major issue when developing new hardware for ML, and the question we should always be asking: **how is the device programmed?**
- To be useful for DNN applications, we should be able to map a high-level program written in something like TensorFlow / PyTorch / MxNet / Keras to run on the hardware.
 - Ideal scenario: users don't need to reason about the hardware that their code is running on. They just see the speedup.
 - Currently: users still need to give *some* thought to the hardware.
 - E.g. if the accelerator uses low-precision arithmetic, the performance of an algorithm in full precision on a CPU may differ from the performance of the algorithm in low-precision on the accelerator.
 - But it is possible to get TensorFlow/PyTorch code to run on an accelerator with minimal modifications!



- The only real distinction between GPUs and ML accelerators is that GPUs weren't originally designed for AI/ML.
 - But there's nothing in the architecture that separates GPUs from all purpose-built ML accelerators.
- ...although individual accelerators do usually have features that GPUs lack.
- As ML tasks capture more of the market for GPUs, GPU designers have been adjusting their architectures to fit ML applications.
 - For example, by supporting low-precision arithmetic.
- As GPU architectures become more specialized to AI tasks, it becomes more accurate to think of them as ML accelerators.

- All computer processors are basically integrated circuit: electronic circuits etched on a single piece of silicon.
- Usually this circuit is fixed when the chip is designed.
- A **field-programmable gate array** or FPGA is a type of chip that allows the end-user to reconfigure the circuit it computes in the field (hence the name).

FPGA as ML accelerator (continued)

FPGAs consist of an array of programmable circuits that can each individually do a small amount of computation, as well as a programmable interconnect that connects these circuits together. The large number of programmable gates in the FPGA makes it a **naturally highly parallel device**.

- You can program it by specifying the circuit you want it to compute in terms of logic gates: basic AND, OR, NOT, etc.
 - Although in practice higher-level languages like Verilog are used.
- This doesn't actually involve physical changes to the circuit that's actually etched on the physical silicon of the FPGA: that's fixed. Rather, the FPGA constructs a logical circuit that is reconfigurable.
 - FPGAs were used historically for circuit simulation.

Why would we want to use an FPGA instead of a GPU/GPU?

- An important property of FPGAs that distinguishes them from CPUs/GPUs: you can choose to have data flow through the chip however you want!
 - Unlike CPUs which are tied to their cache-hierarchy-based data model, or GPUs which perform best under a streaming model.
- FPGAs often use less power to accomplish the same work compared with other architectures.
 - But they are also typically slower.

Why would we want to use an FPGA instead of an ASIC?

- Pro for FPGA: Much cheaper to program and FPGA than the design an ASIC.
- Pro for FPGA: A single FPGA costs much less than the first ASIC you synthesize.
- Pro for FPGA: FPGA designs can be adjusted on the fly.
- Pro for ASIC: The marginal cost of producing an additional ASIC is lower if you really want to synthesize millions or billions of them.
- Pro for ASIC: ASICs can typically achieve higher speed and lower power.

Users of FPGAs for Machine Learning

e.g. Microsoft's Project Capatult/Project Brainwave.

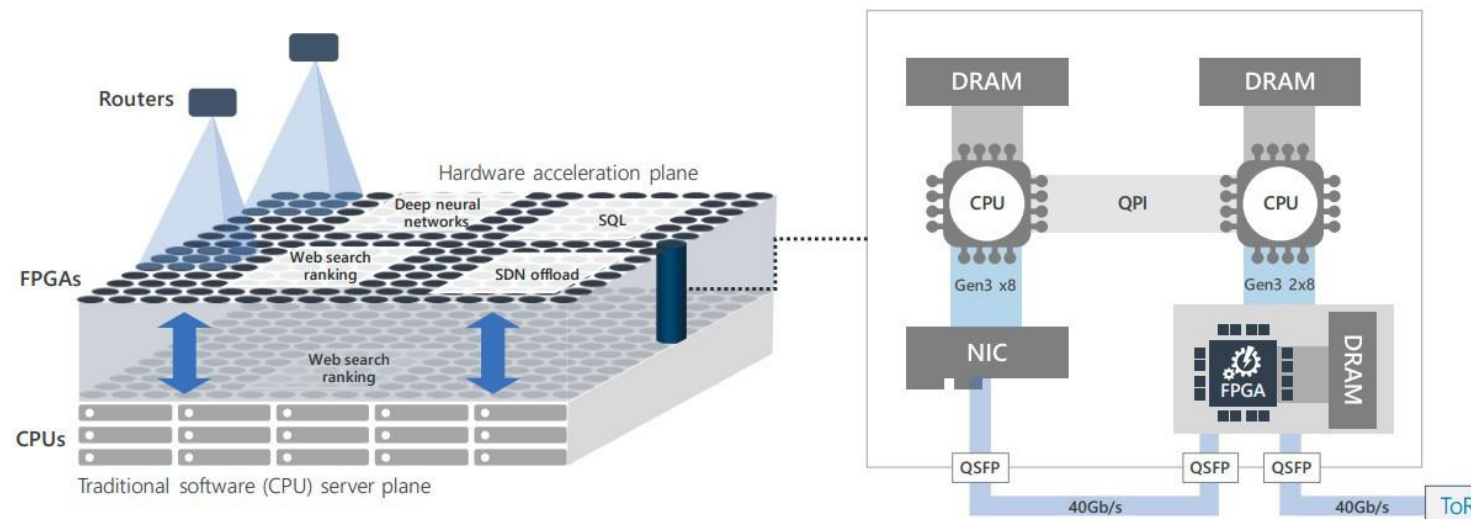


Figure 1. The first generation of Catapult-enhanced servers in production (right) consists of dual Xeon CPUs with a PCIe-attached FPGA. Each FPGA sits in-line between the 40Gbps server NIC and the TOR, enabling in-situ processing of network packets and point-to-point connectivity with up to hundreds of thousands of other FPGAs at datacenter scale. Multiple FPGAs can be allocated as a single shared hardware microservice with no software in the loop (left), enabling scalable workloads and better load balancing between CPUs and FPGAs.

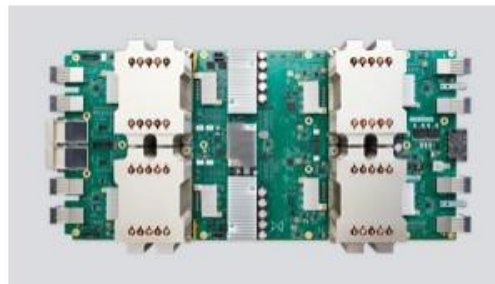
An example of a designed-for-ML accelerator: The Tensor Processing Unit (TPU)

- Google's Tensor Processing Unit (TPU) in 2015 was one of the first specialized architectures for machine learning and AI applications.
- The original version focused on **fast inference via high-throughput 8-bit arithmetic.**
- Most of the chip is dedicated to **accelerating 8-bit integer dense-matrix-dense-matrix multiplies**
 - Note that even though the numbers it multiplies are in 8-bit, it uses 32-bit accumulators to sum up the results.
 - This larger accumulator is common in ML architectures that use low precision.
- ...with a little bit of logic on the side to apply activation functions. **The second- and third-generation TPUs are designed to also support training and can calculate in float/bfloat16.**



Now ML accelerators are found in the cloud!

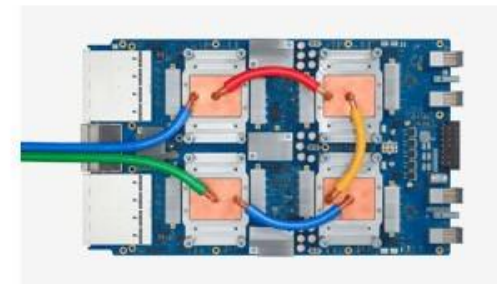
Cloud TPU offering



Cloud TPU v2

180 teraflops

64 GB High Bandwidth Memory (HBM)



Cloud TPU v3

420 teraflops

128 GB HBM

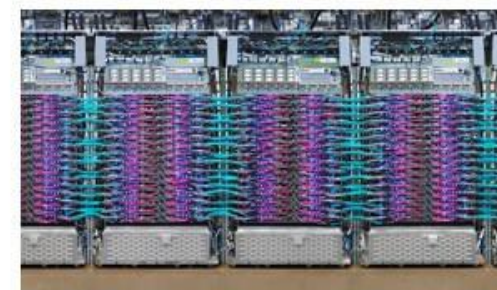


Cloud TPU v2 Pod

11.5 petaflops

4 TB HBM

2-D toroidal mesh network



Cloud TPU v3 Pod

100+ petaflops

32 TB HBM

2-D toroidal mesh network

How do we program a TPU

- Essentially the same as for a GPU
- Google supports running ML kernels on TPUs.
 - A TPU is just another accelerator the ML framework can compute on and store data on
 - This makes it (relatively) easy to train and infer deep neural networks using tools you're already familiar with.

- Recent systems-on-a-chip developed for personal devices (laptops, phones) now tend to include AI accelerators.
 - You can think of this as like a CPU+GPU combo
- Examples include the M1 and M2 chips from Apple.

M2_chip_layout.png

How to choose your accelerator?

- Different accelerators provide different trade-offs
- Need to ask yourself some questions
 - Am I running training or inference?
 - What is my budget?
 - Where am I running? Am I limited to one cloud provider? Am I running on an edge device?
 - What did other people with similar tasks use?
 - Do I care more about throughput or latency? Or energy?

Data-parallel training with GPUs

Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

Devices

.cuda()

GPU1

θ



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

batch

x

...

Devices

.cuda()

.cuda()

GPU1

θ

x

$\Rightarrow$

forward pass

$\Rightarrow$

backward pass

$\Rightarrow$

$\nabla_{\theta} L(x_1)$



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

batch

x

...

prepare next batch

Devices

.cuda()

.cuda()

new model

GPU1

θ

x

forward pass

backward pass

$\nabla_{\theta} L(x_1)$

step

θ



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

Devices

replicate

GPU1

θ

GPU2

θ

GPU3

θ

GPU4

θ

Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

batch

x_1 x_2 x_3 x_4

Devices

replicate

scatter

GPU1

θ

x_1

GPU2

θ

x_2

GPU3

θ

x_3

GPU4

θ

x_4

Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

batch

x_1

x_2

x_3

x_4

...

Devices

replicate

scatter

GPU1

θ

x_1

forward pass

backward pass

$\nabla_{\theta} L(x_1)$

GPU2

θ

x_2

forward pass

backward pass

$\nabla_{\theta} L(x_2)$

GPU3

θ

x_3

forward pass

backward pass

$\nabla_{\theta} L(x_3)$

GPU4

θ

x_4

forward pass

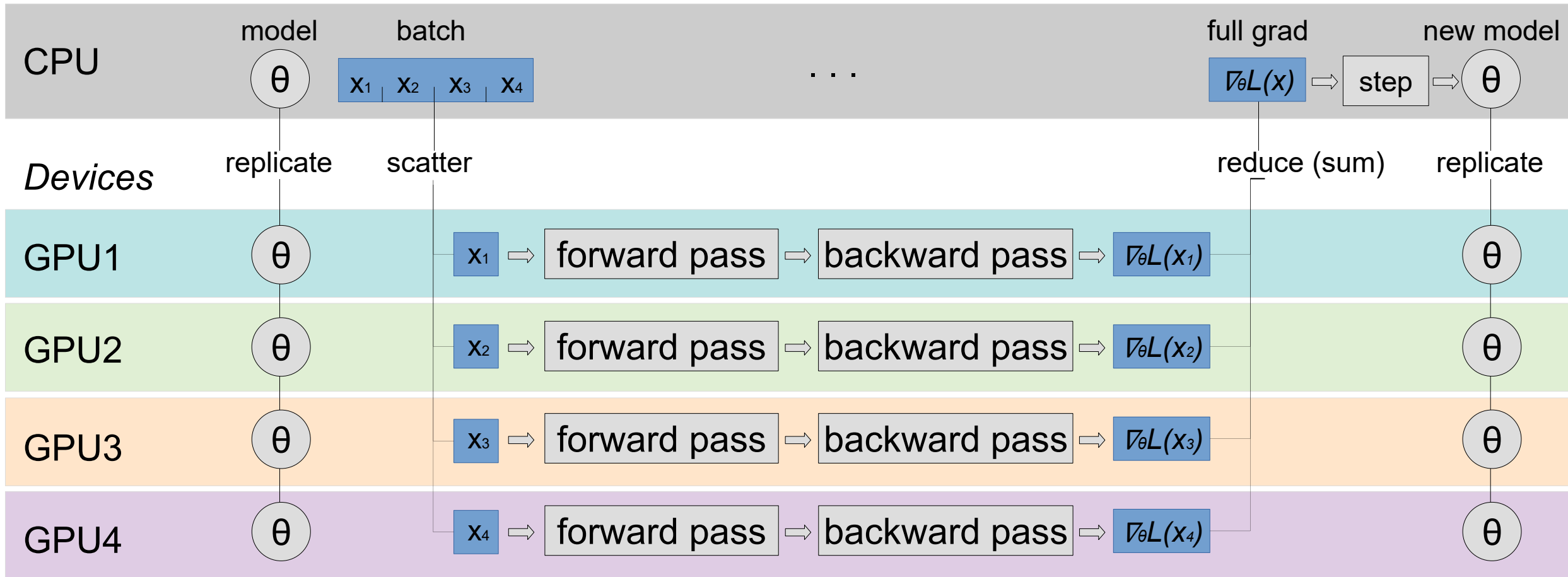
backward pass

$\nabla_{\theta} L(x_4)$

Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

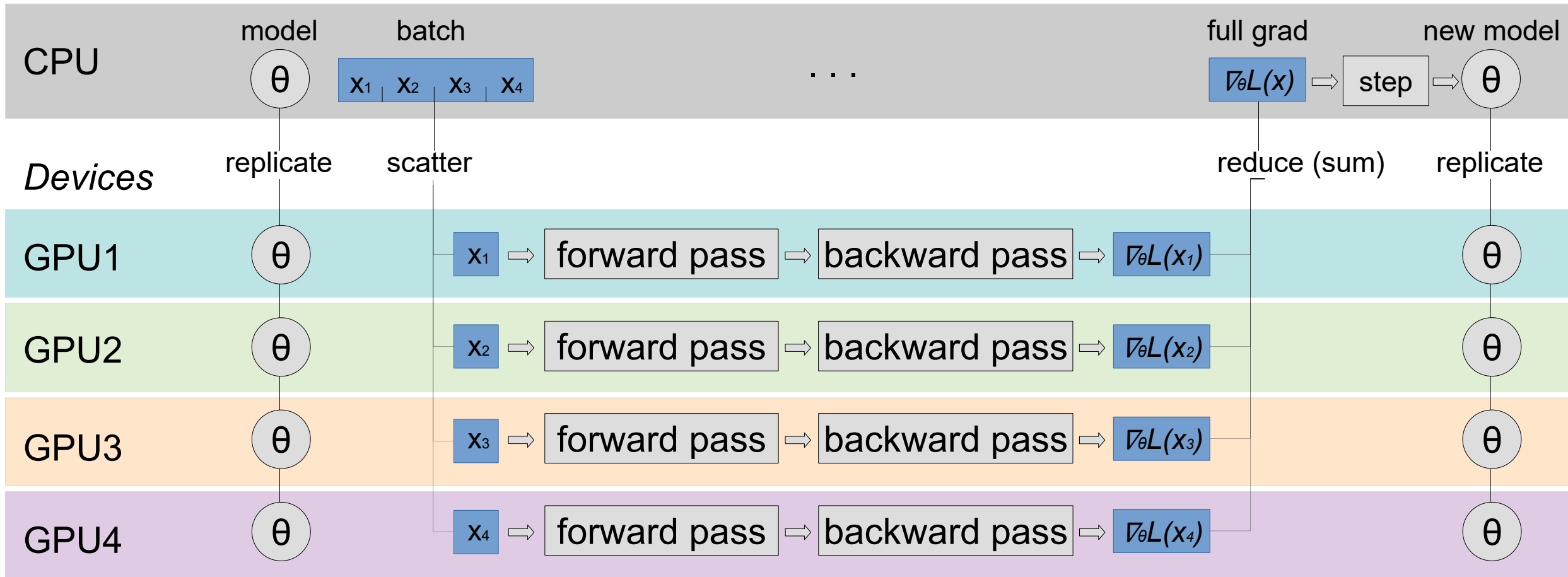
Host



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

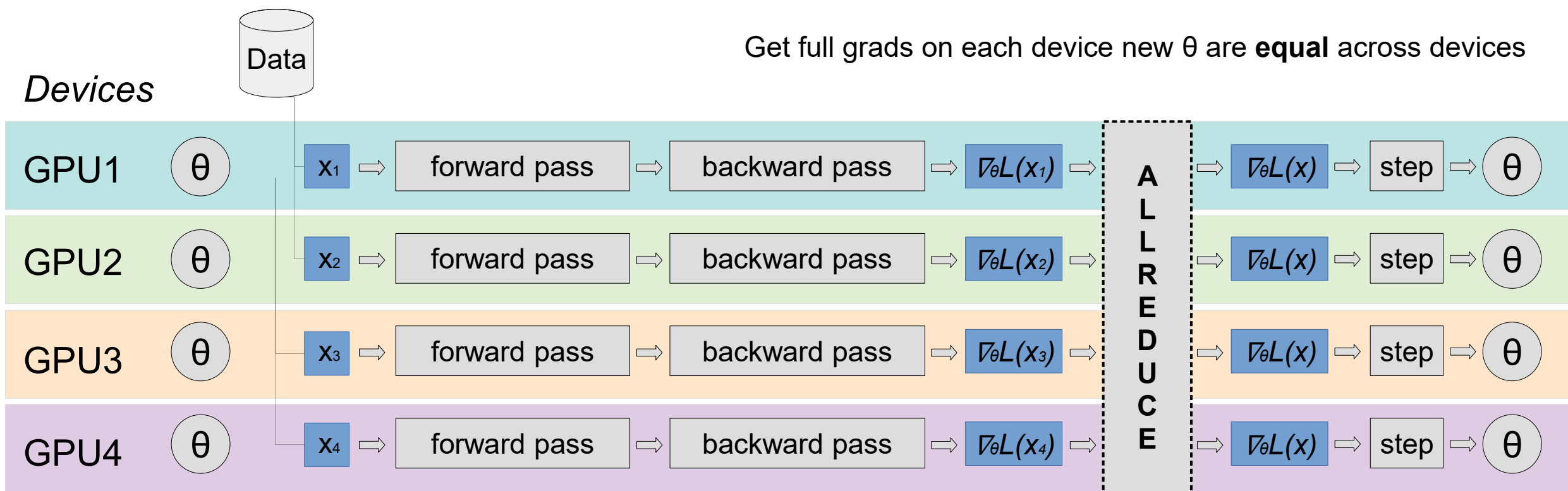


All-Reduce data parallel

arxiv.org/abs/1706.02677

Idea: get rid of the host, each gpu runs its own computation

Q: why will weights be equal after such step?

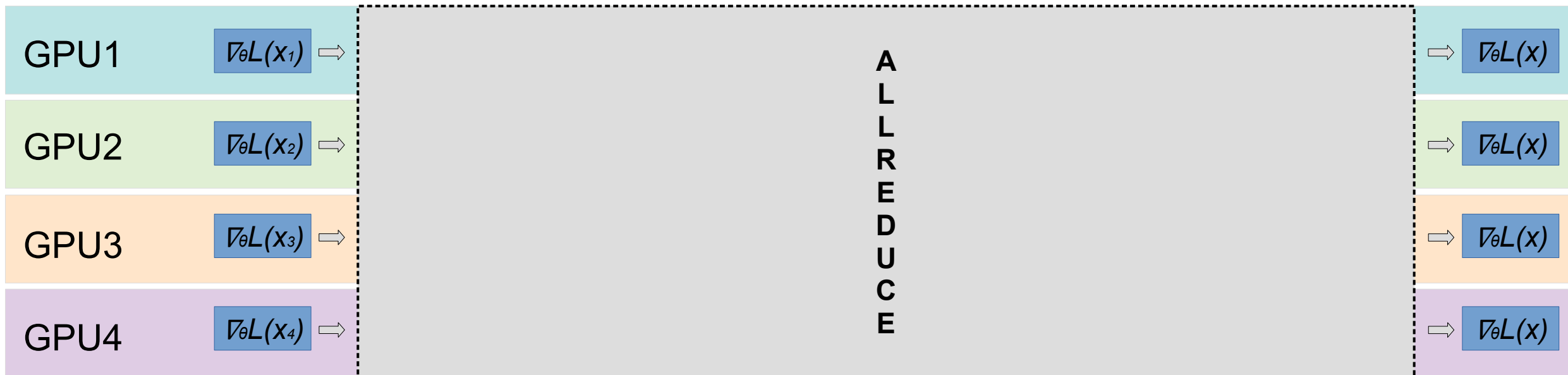


Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Devices



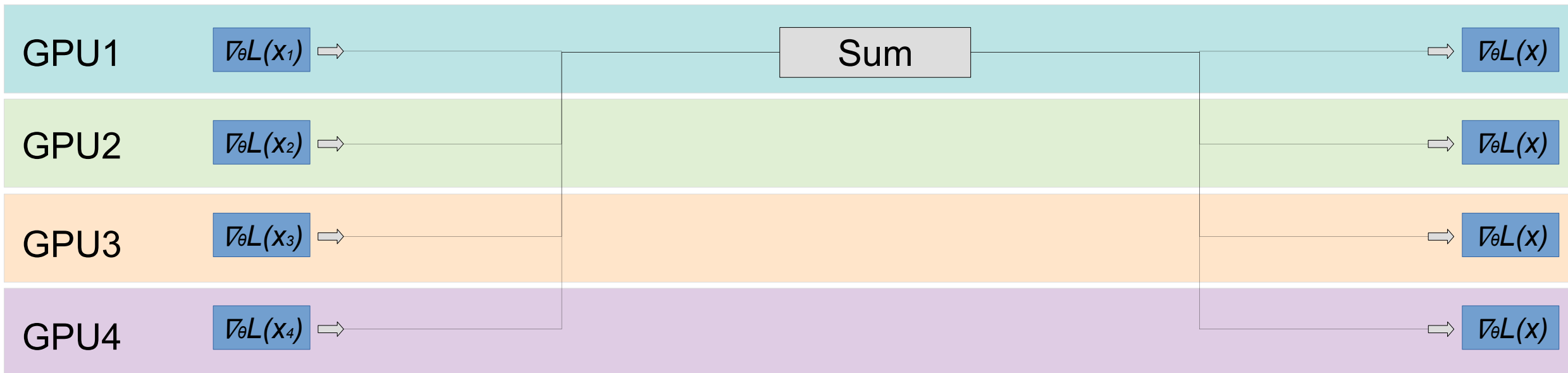
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Naive implementation

Devices



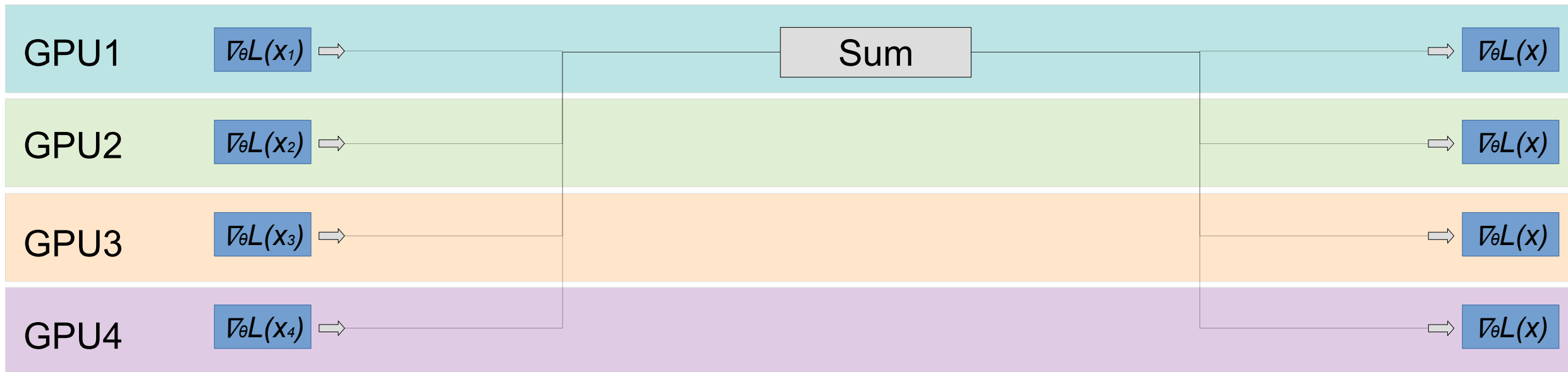
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Q: Can we do better?

Devices



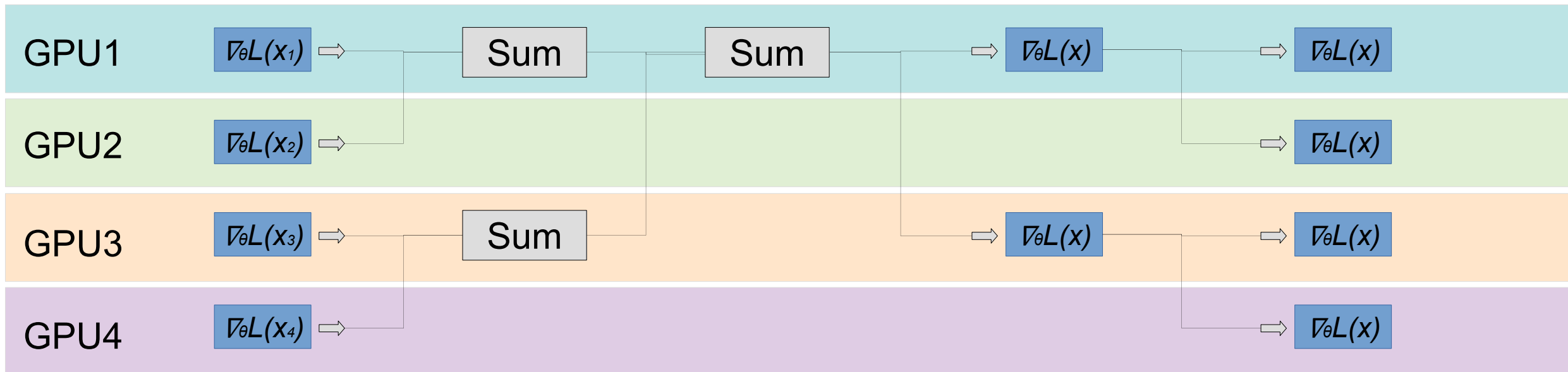
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Tree-allreduce

Devices



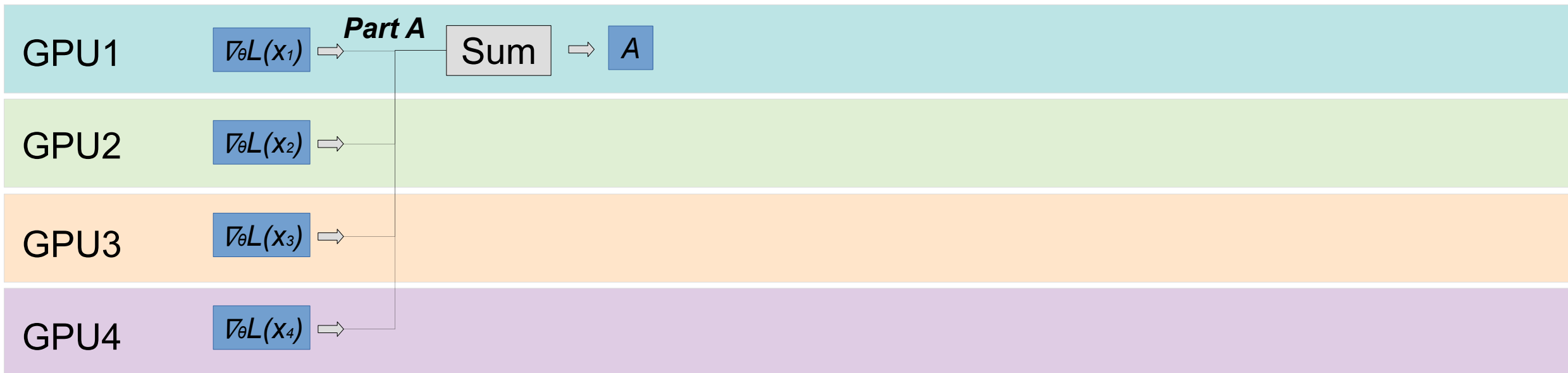
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Butterfly-allreduce – split data into chunks (ABCD)

Devices



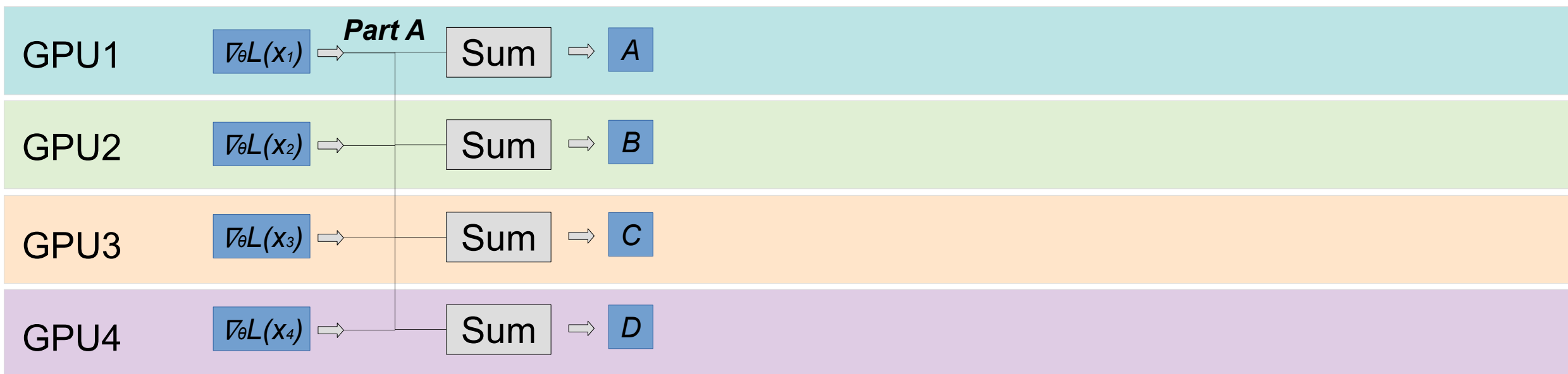
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Butterfly-allreduce – split data into chunks (ABCD)

Devices



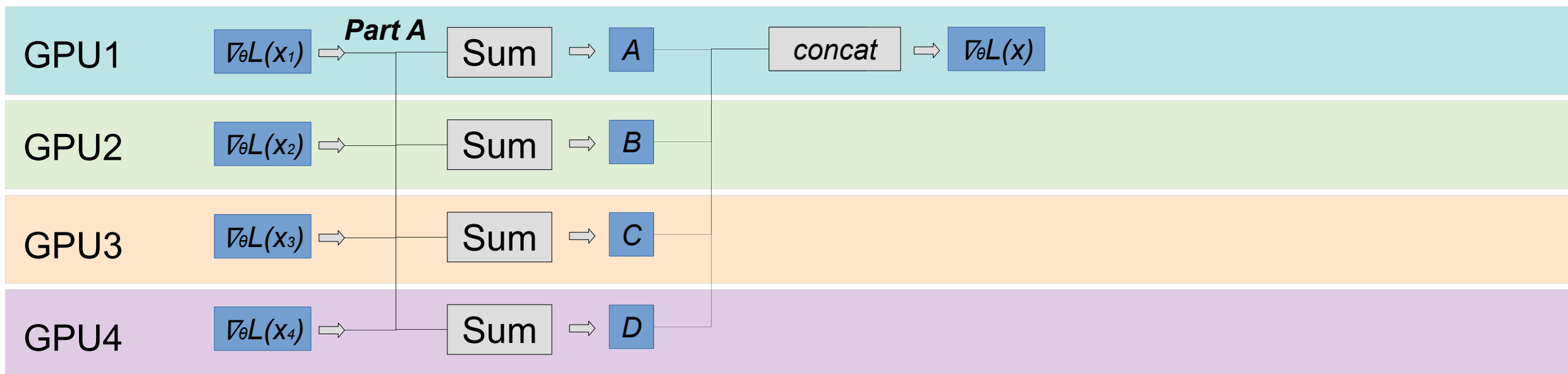
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Butterfly-allreduce – split data into chunks (ABCD)

Devices



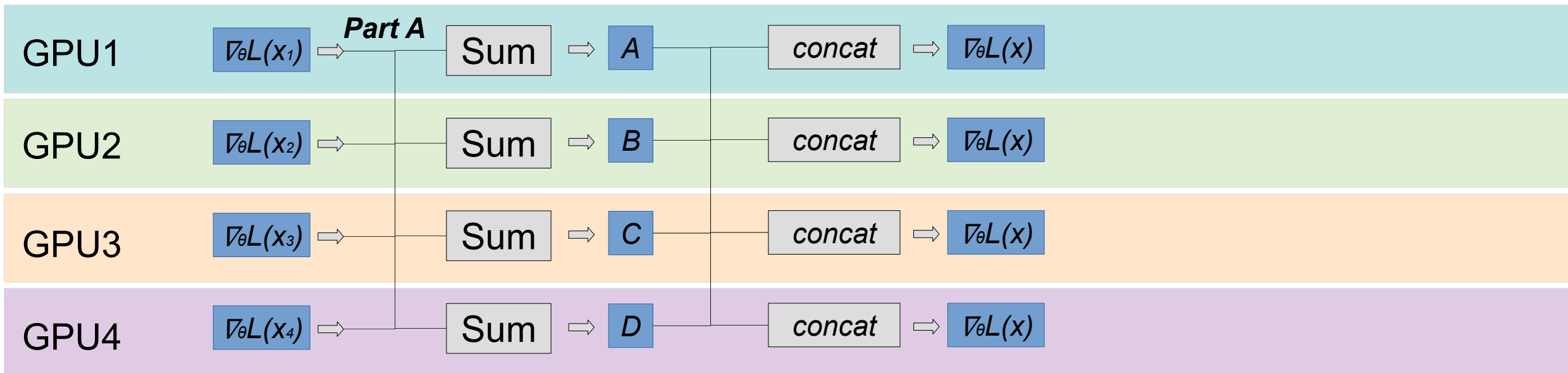
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

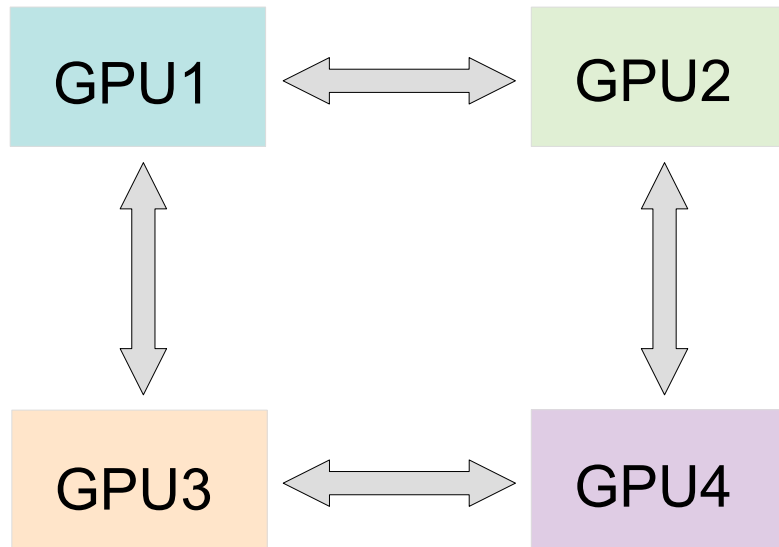
Ring-allreduce – split data into chunks (ABCD)

Devices



Ring allreduce

Bonus quest: you can only send data between **adjacent** gpus



Ring topology



Image: [graphcore](#) IPU server

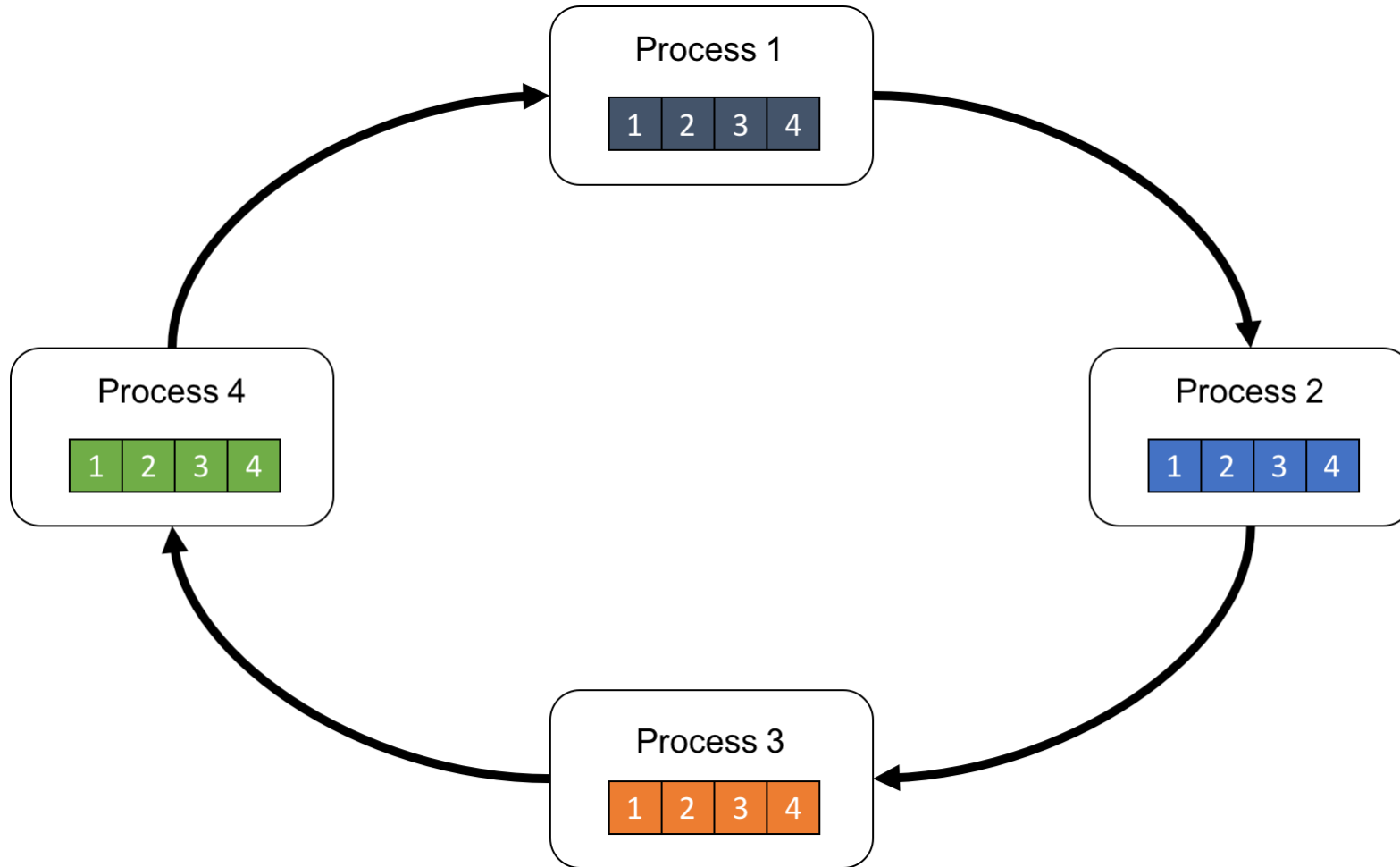
Read more: <https://tech.preferred.jp/en/blog/technologies-behind-distributed-deep-learning-allreduce/>

Ring allreduce

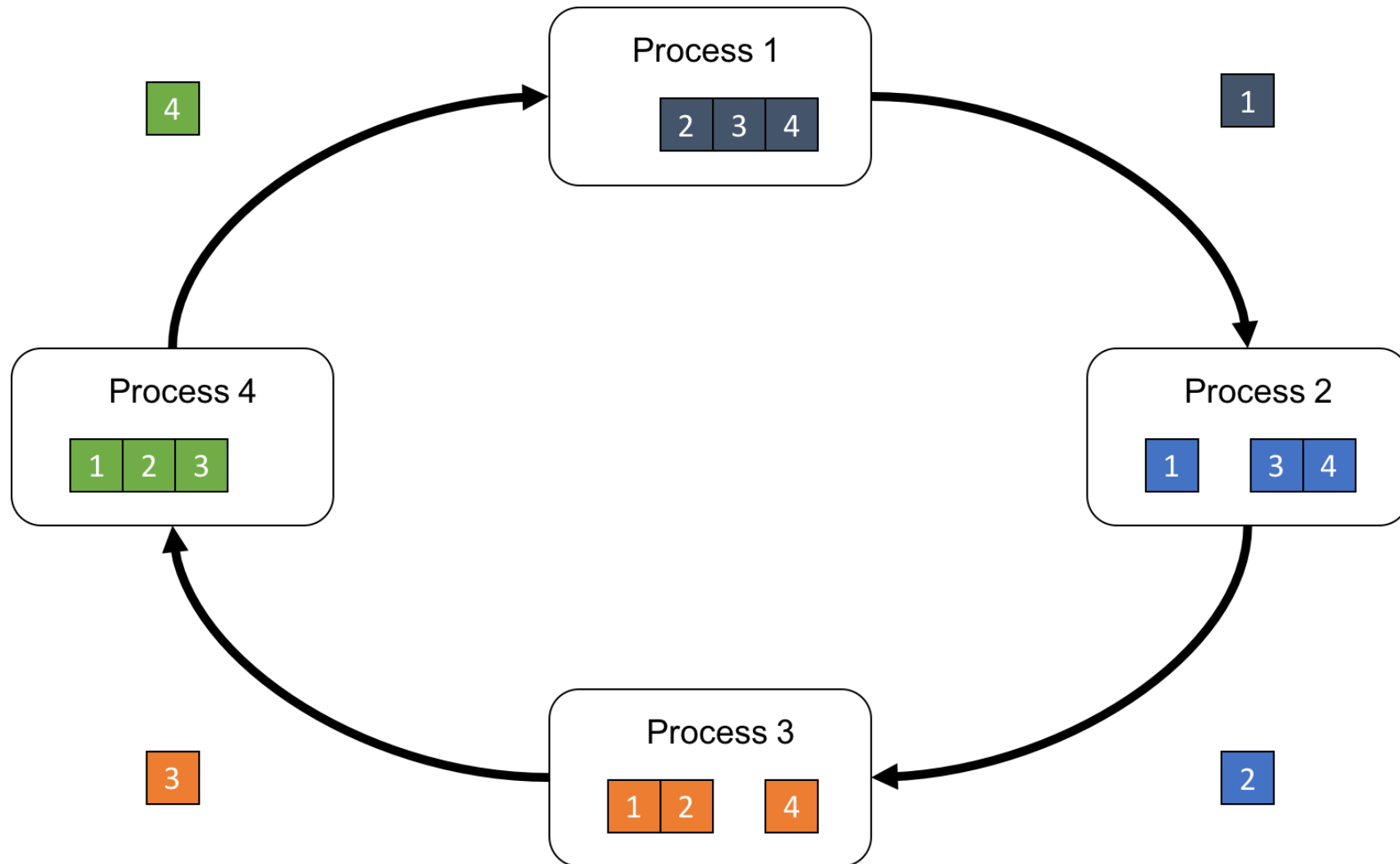
Bonus quest: you can only send data between **adjacent** gpus



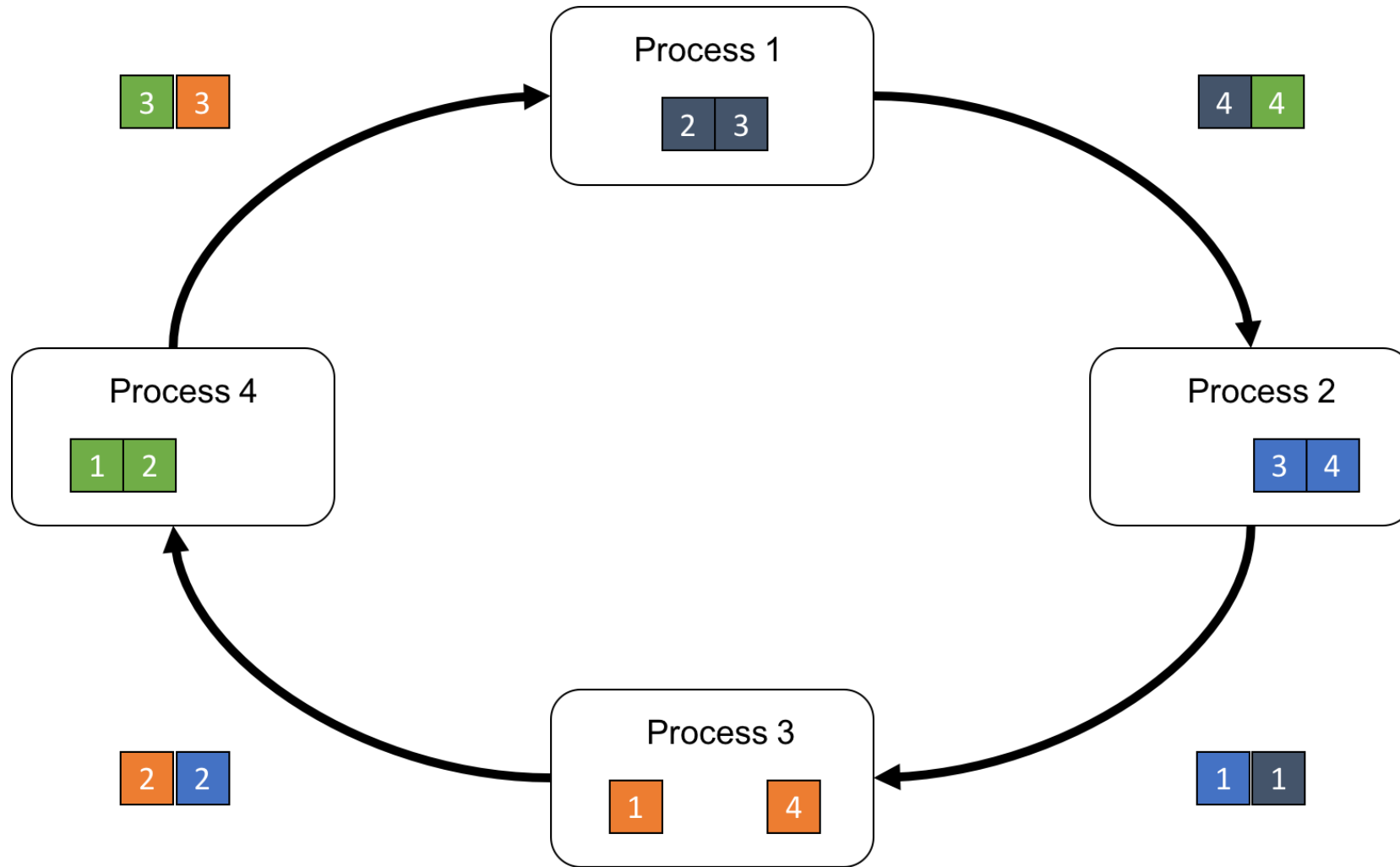
Ring allreduce



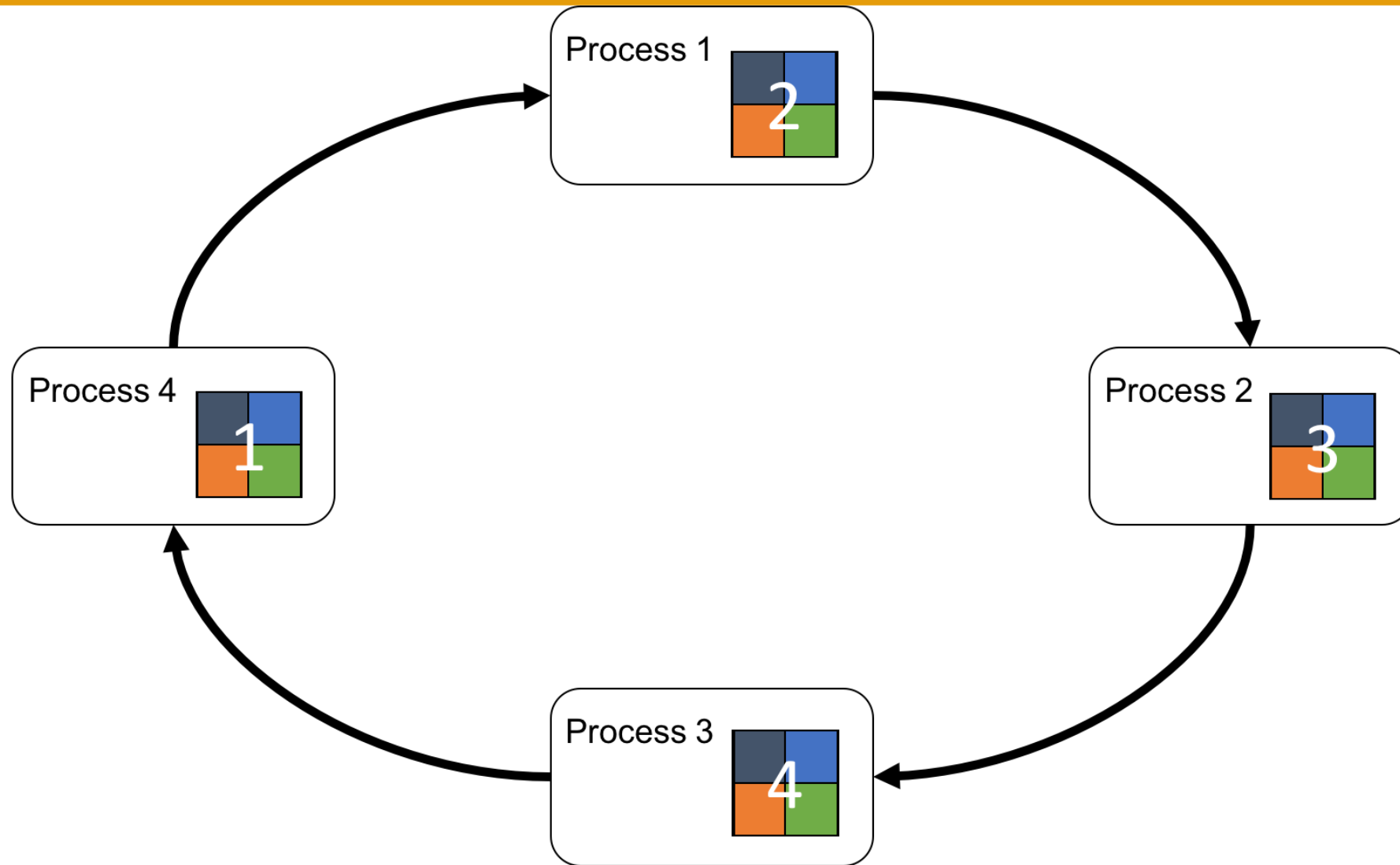
Ring allreduce



Ring allreduce



Ring allreduce

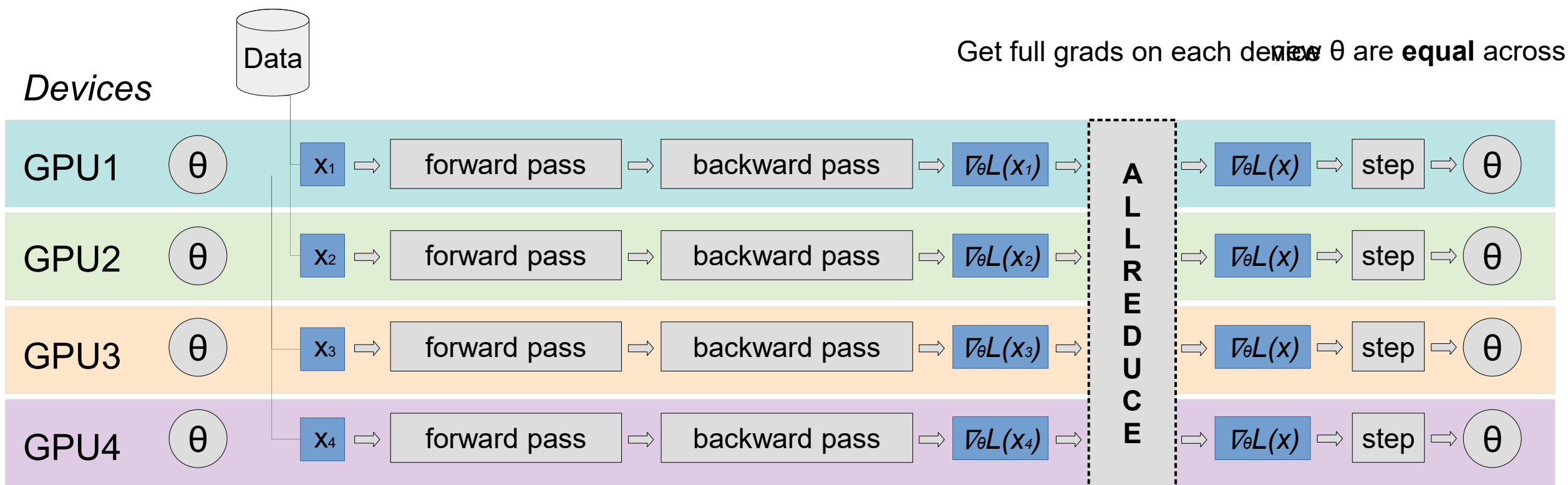


All-Reduce data parallel

arxiv.org/abs/1706.02677

Idea: get rid of the host, each gpu runs its own computation

Q: why will weights be equal after such step?

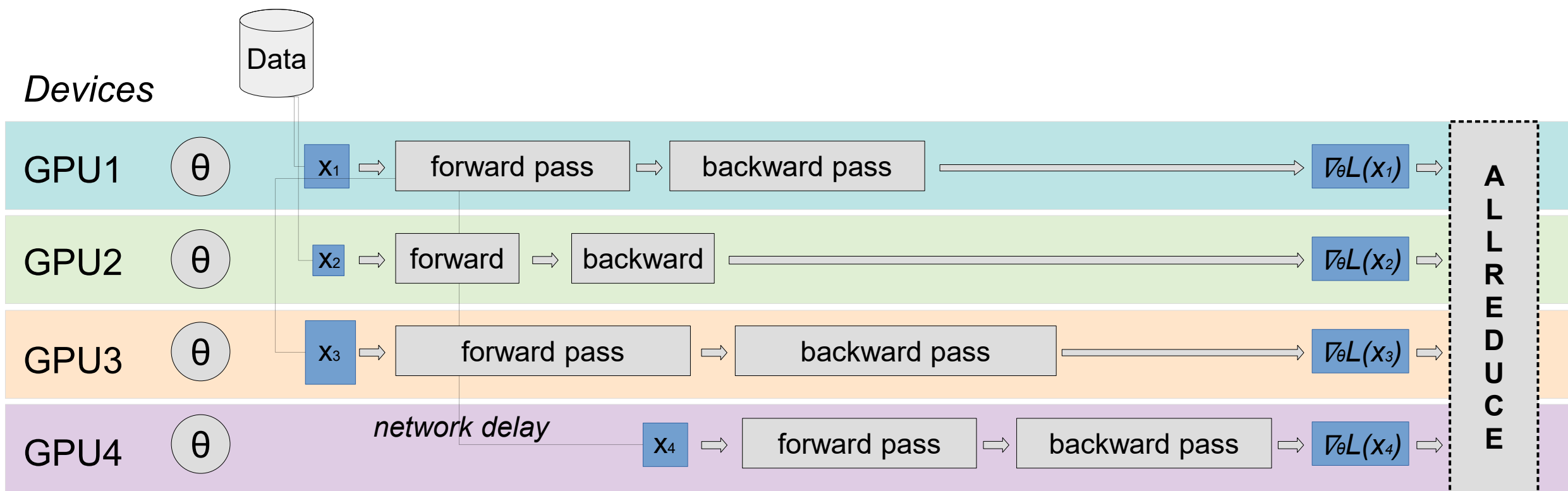


All-Reduce data parallel VS reality

arxiv.org/abs/1706.02677

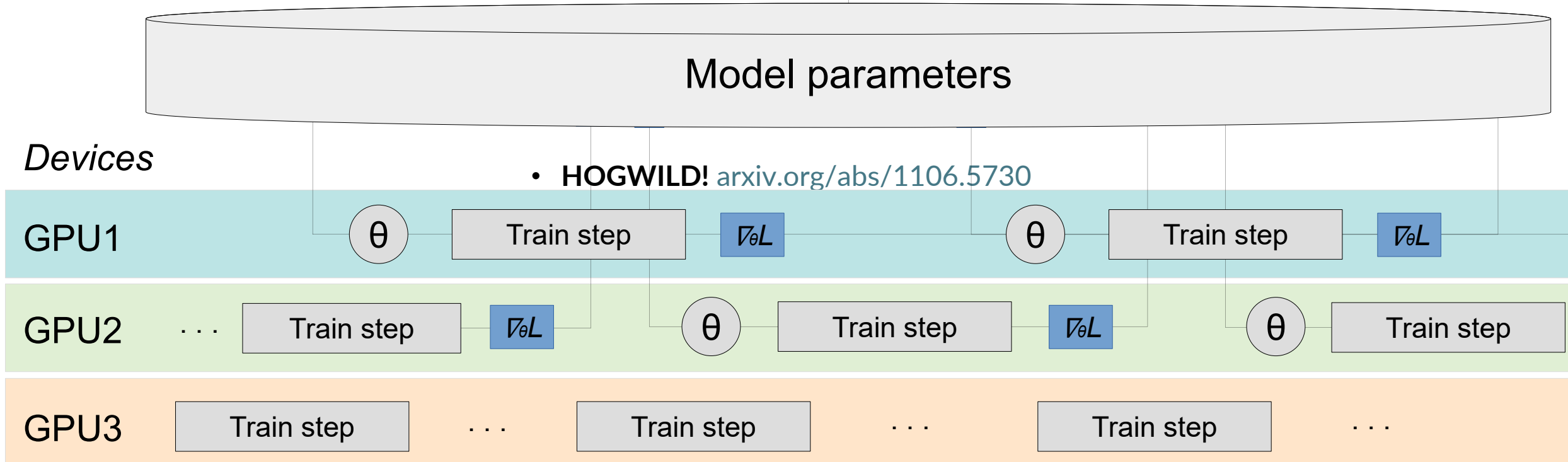
Each gpu has different processing time & delays

Q: can we improve device utilization?



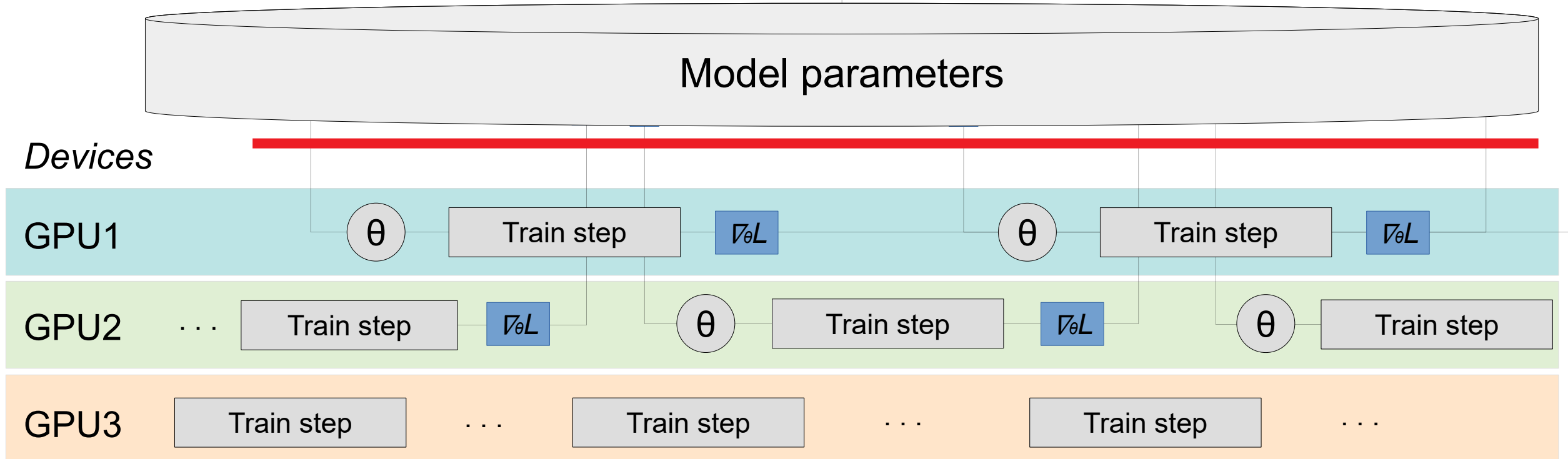
Recap: Parameter Server

Idea: remove synchronization step altogether, use parameter server



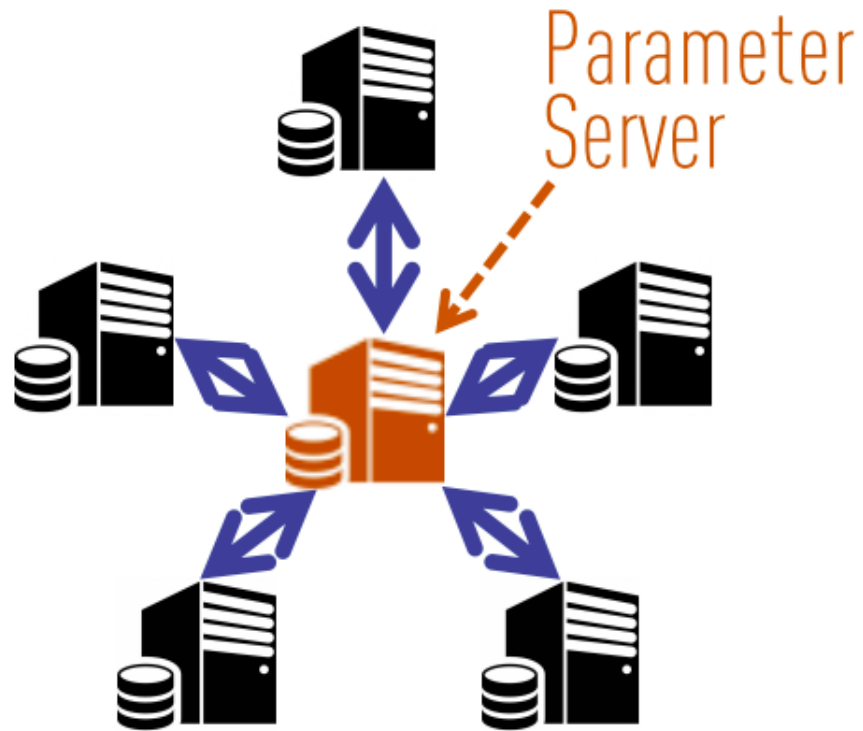
Recap: Parameter Server

Idea: remove synchronization step altogether, use parameter server

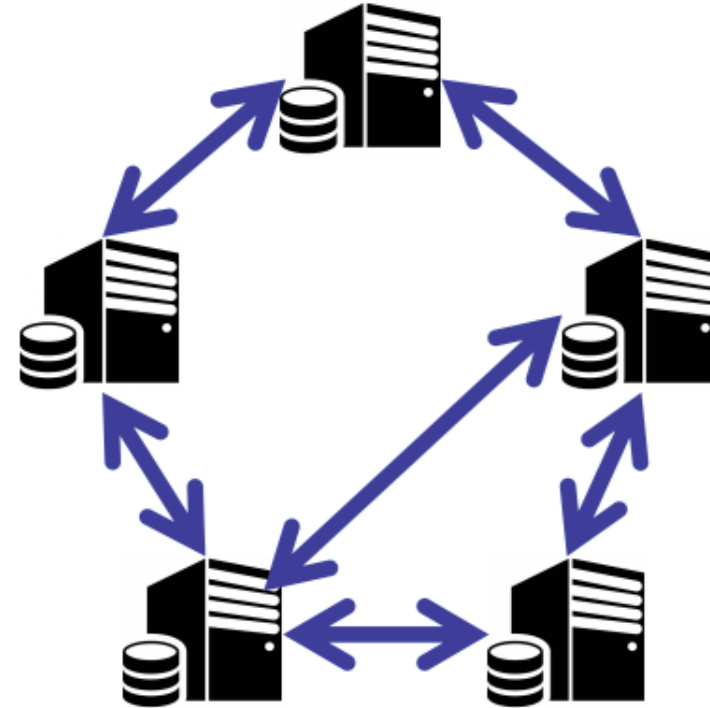


Problem: parameter servers need to ingest tons of data over training

Decentralized Training with Gossip



(a) Centralized Topology



(b) Decentralized Topology

Decentralized Training with Gossip

Source: <https://tinyurl.com/can-dsgd-outperform>

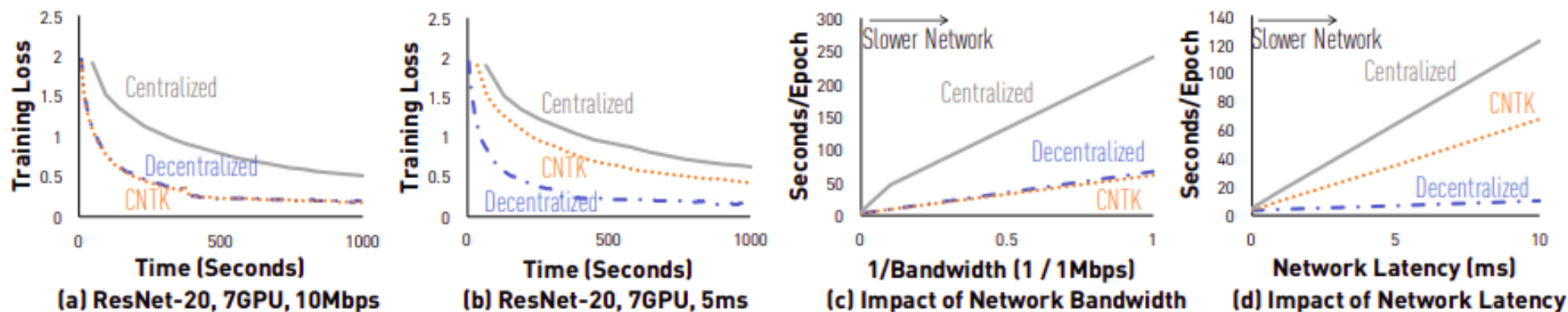


Figure 2: Comparison between D-PSGD and two centralized implementations (7 and 10 GPUs).

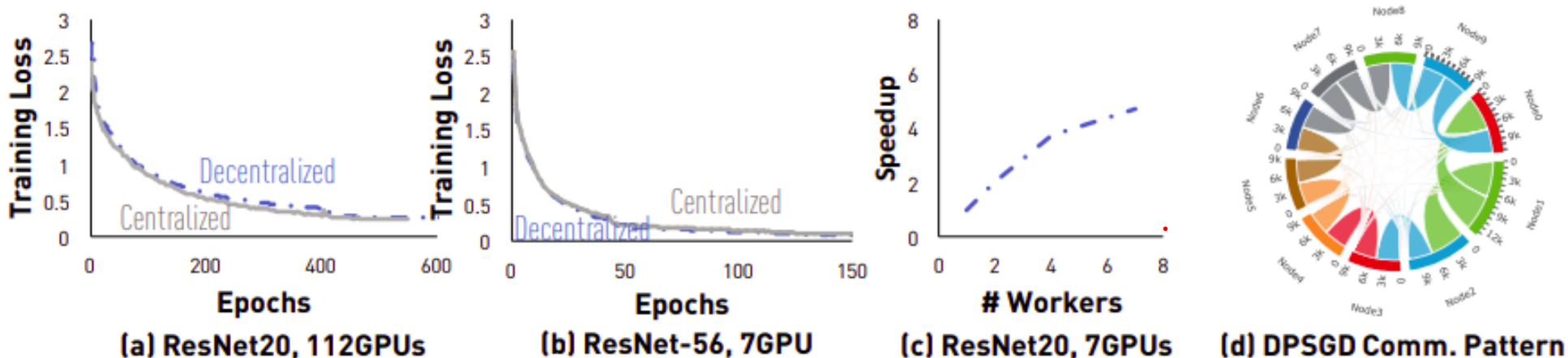
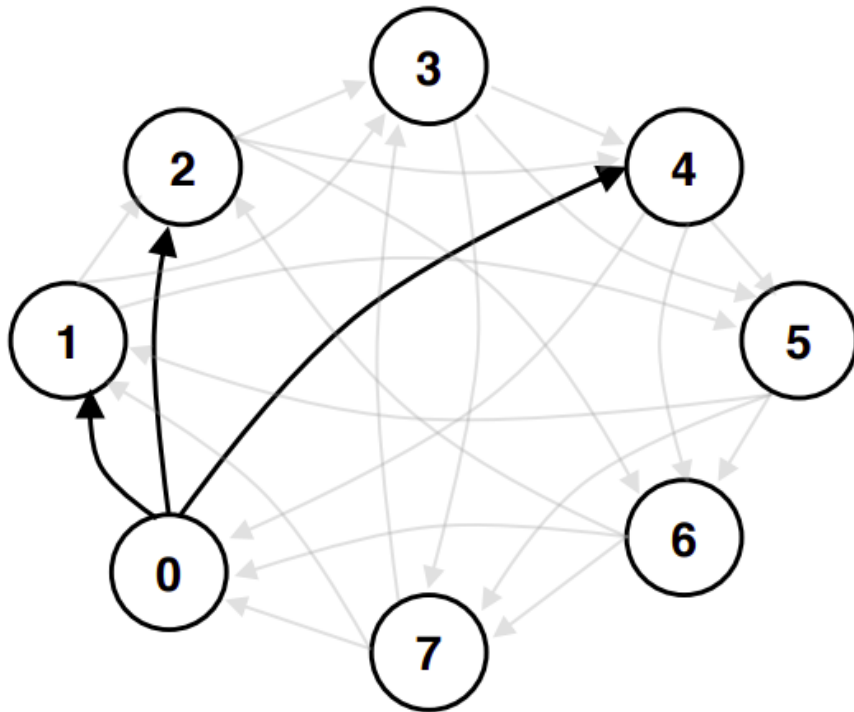


Figure 3: (a) Convergence Rate; (b) D-PSGD Speedup; (c) D-PSGD Communication Patterns.

Stochastic Gradient Push

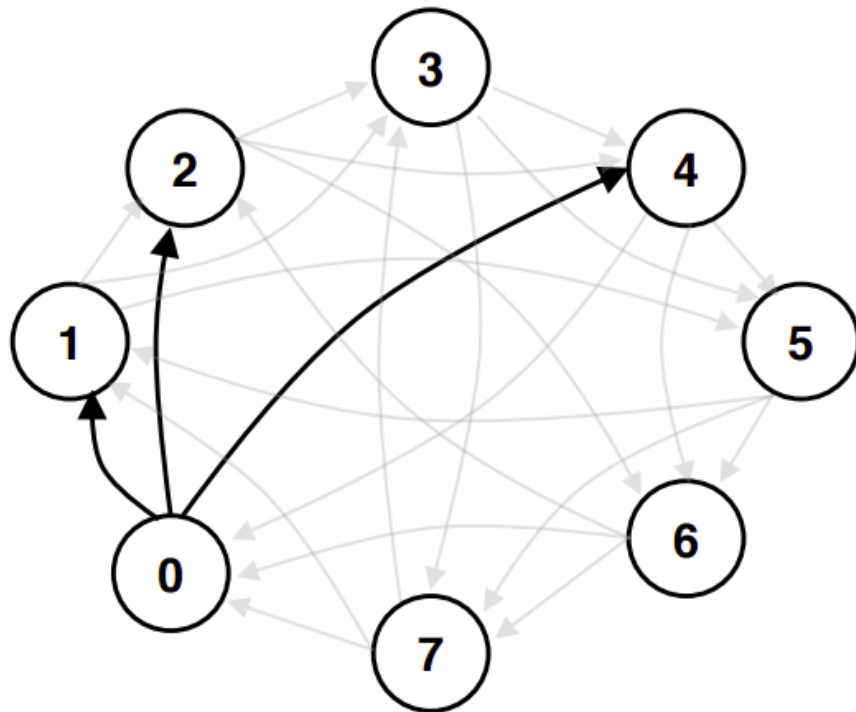
Source: <https://arxiv.org/abs/1811.10792>



(a) Directed Exponential Graph highlighting node 0's out-neighbours

Stochastic Gradient Push

Source: <https://arxiv.org/abs/1811.10792>



(a) Directed Exponential Graph highlighting node 0's out-neighbours

Algorithm 1 Stochastic Gradient Push (SGP)

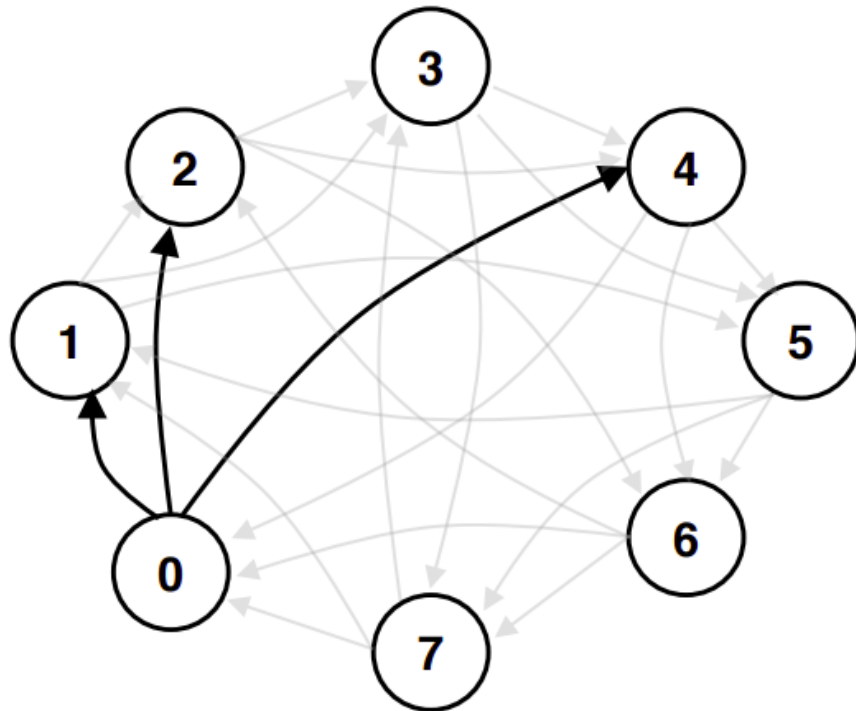
Require: Initialize $\gamma > 0$, $\mathbf{x}_i^{(0)} = \mathbf{z}_i^{(0)} \in \mathbb{R}^d$ and $w_i^{(0)} = 1$ for all nodes $i \in \{1, 2, \dots, n\}$

- 1: **for** $k = 0, 1, 2, \dots, K$, at node i , **do**
- 2: Sample new mini-batch $\xi_i^{(k)} \sim \mathcal{D}_i$ from local distribution
- 3: Compute mini-batch gradient at $\mathbf{z}_i^{(k)}$: $\nabla F_i(\mathbf{z}_i^{(k)}; \xi_i^{(k)})$

<to be continued>

Stochastic Gradient Push

Source: <https://arxiv.org/abs/1811.10792>



(a) Directed Exponential Graph highlighting node 0's out-neighbours

Algorithm 1 Stochastic Gradient Push (SGP)

Require: Initialize $\gamma > 0$, $\mathbf{x}_i^{(0)} = \mathbf{z}_i^{(0)} \in \mathbb{R}^d$ and $w_i^{(0)} = 1$ for all nodes $i \in \{1, 2, \dots, n\}$

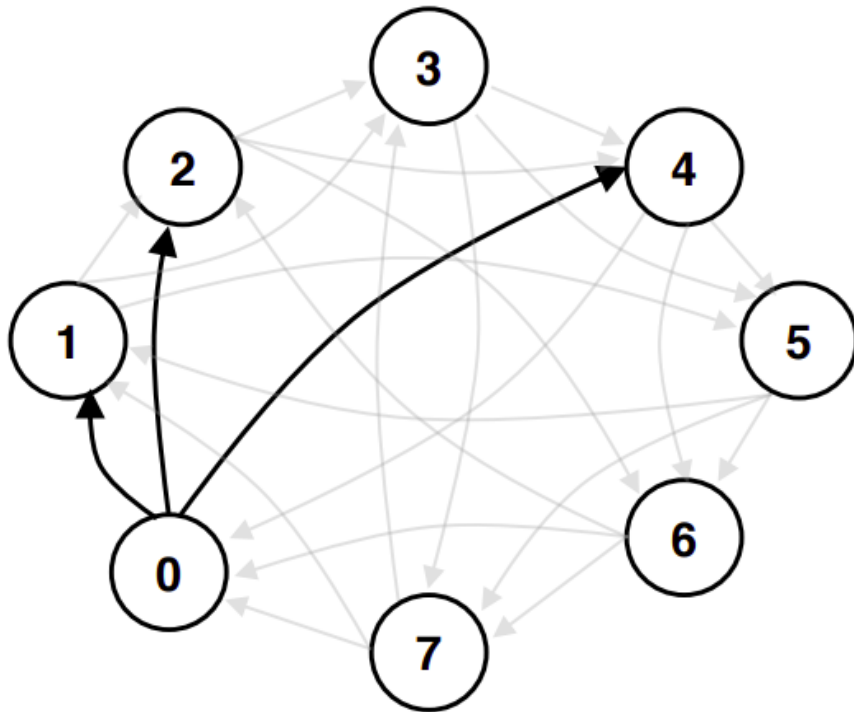
- 1: **for** $k = 0, 1, 2, \dots, K$, at node i , **do**
- 2: Sample new mini-batch $\xi_i^{(k)} \sim \mathcal{D}_i$ from local distribution
- 3: Compute mini-batch gradient at $\mathbf{z}_i^{(k)}$: $\nabla F_i(\mathbf{z}_i^{(k)}; \xi_i^{(k)})$
- 4: $\mathbf{x}_i^{(k+\frac{1}{2})} = \mathbf{x}_i^{(k)} - \gamma \nabla F_i(\mathbf{z}_i^{(k)}; \xi_i^{(k)})$

normal GD step

<to be continued>

Stochastic Gradient Push

Source: <https://arxiv.org/abs/1811.10792>



(a) Directed Exponential Graph highlighting node 0's out-neighbours

Algorithm 1 Stochastic Gradient Push (SGP)

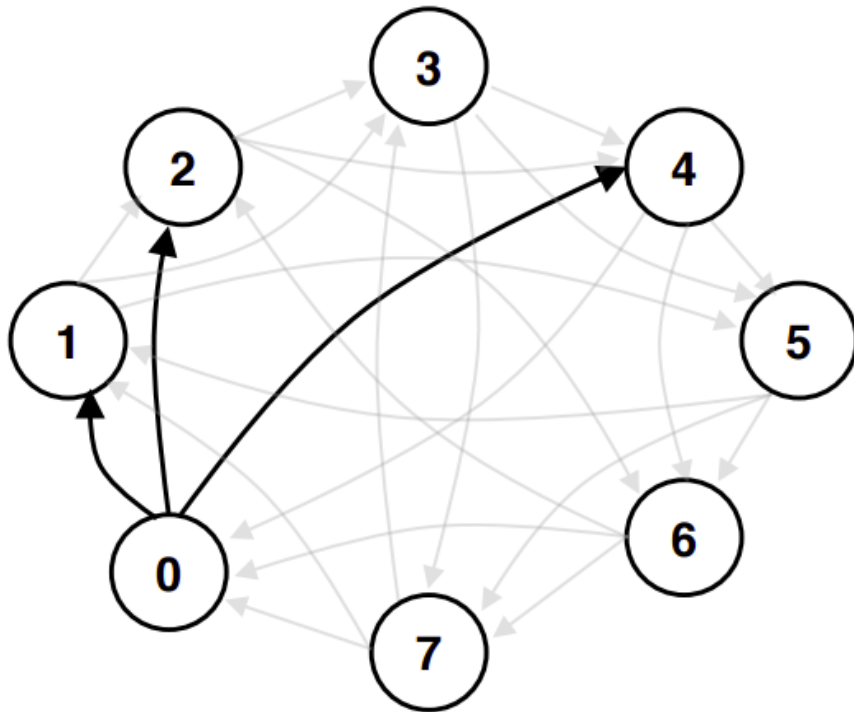
Require: Initialize $\gamma > 0$, $\mathbf{x}_i^{(0)} = \mathbf{z}_i^{(0)} \in \mathbb{R}^d$ and $w_i^{(0)} = 1$ for all nodes $i \in \{1, 2, \dots, n\}$

- 1: **for** $k = 0, 1, 2, \dots, K$, at node i , **do**
- 2: Sample new mini-batch $\xi_i^{(k)} \sim \mathcal{D}_i$ from local distribution
- 3: Compute mini-batch gradient at $\mathbf{z}_i^{(k)}$: $\nabla \mathbf{F}_i(\mathbf{z}_i^{(k)}; \xi_i^{(k)})$
- 4: $\mathbf{x}_i^{(k+\frac{1}{2})} = \mathbf{x}_i^{(k)} - \gamma \nabla \mathbf{F}_i(\mathbf{z}_i^{(k)}; \xi_i^{(k)})$
- 5: Send $(p_{j,i}^{(k)} \mathbf{x}_i^{(k+\frac{1}{2})}, p_{j,i}^{(k)} w_i^{(k)})$ to out-neighbors;
 receive $(p_{i,j}^{(k)} \mathbf{x}_j^{(k+\frac{1}{2})}, p_{i,j}^{(k)} w_j^{(k)})$ from in-neighbors

<to be continued>

Stochastic Gradient Push

Source: <https://arxiv.org/abs/1811.10792>



(a) Directed Exponential Graph highlighting node 0's out-neighbours

Algorithm 1 Stochastic Gradient Push (SGP)

Require: Initialize $\gamma > 0$, $\mathbf{x}_i^{(0)} = \mathbf{z}_i^{(0)} \in \mathbb{R}^d$ and $w_i^{(0)} = 1$ for all nodes $i \in \{1, 2, \dots, n\}$

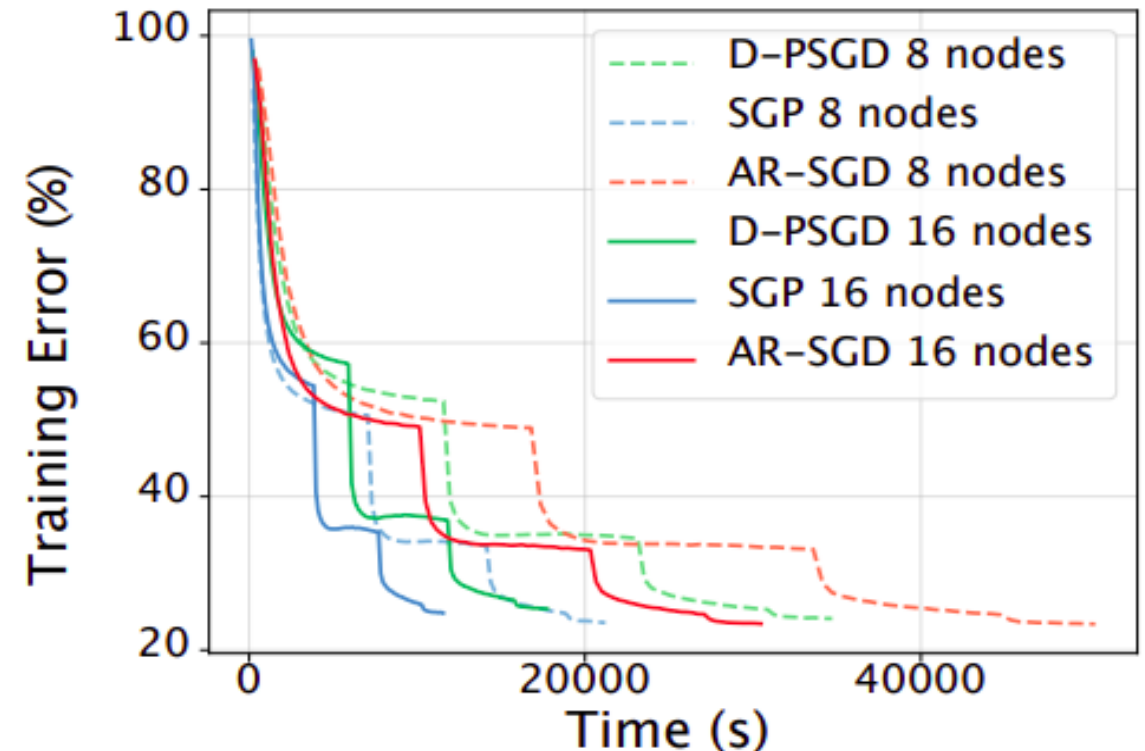
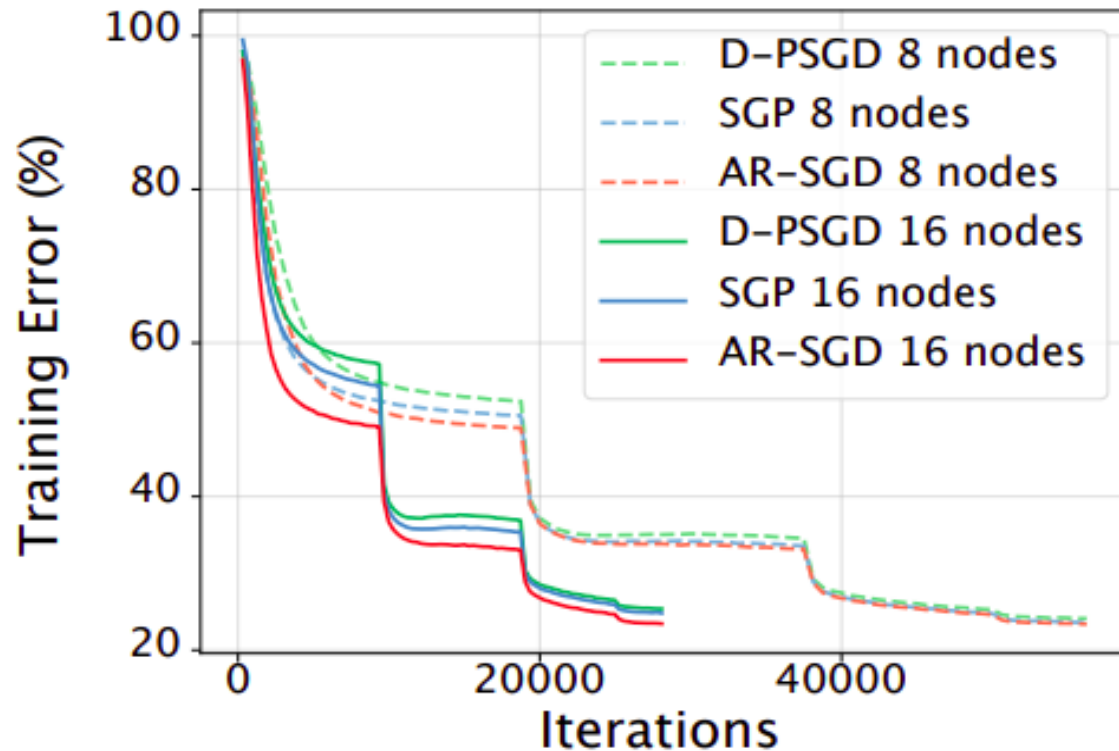
- 1: **for** $k = 0, 1, 2, \dots, K$, at node i , **do**
- 2: Sample new mini-batch $\xi_i^{(k)} \sim \mathcal{D}_i$ from local distribution
- 3: Compute mini-batch gradient at $\mathbf{z}_i^{(k)}$: $\nabla F_i(\mathbf{z}_i^{(k)}; \xi_i^{(k)})$
- 4: $\mathbf{x}_i^{(k+\frac{1}{2})} = \mathbf{x}_i^{(k)} - \gamma \nabla F_i(\mathbf{z}_i^{(k)}; \xi_i^{(k)})$
- 5: Send $(p_{j,i}^{(k)} \mathbf{x}_i^{(k+\frac{1}{2})}, p_{j,i}^{(k)} w_i^{(k)})$ to out-neighbors;
 receive $(p_{i,j}^{(k)} \mathbf{x}_j^{(k+\frac{1}{2})}, p_{i,j}^{(k)} w_j^{(k)})$ from in-neighbors
- 6: $\mathbf{x}_i^{(k+1)} = \sum_j p_{i,j}^{(k)} \mathbf{x}_j^{(k+\frac{1}{2})}$
- 7: $w_i^{(k+1)} = \sum_j p_{i,j}^{(k)} w_j^{(k)}$
- 8: $\mathbf{z}_i^{(k+1)} = \mathbf{x}_i^{(k+1)} / w_i^{(k+1)}$
- 9: **end for**

**weighted
average**

Stochastic Gradient Push

Source: <https://arxiv.org/abs/1811.10792>

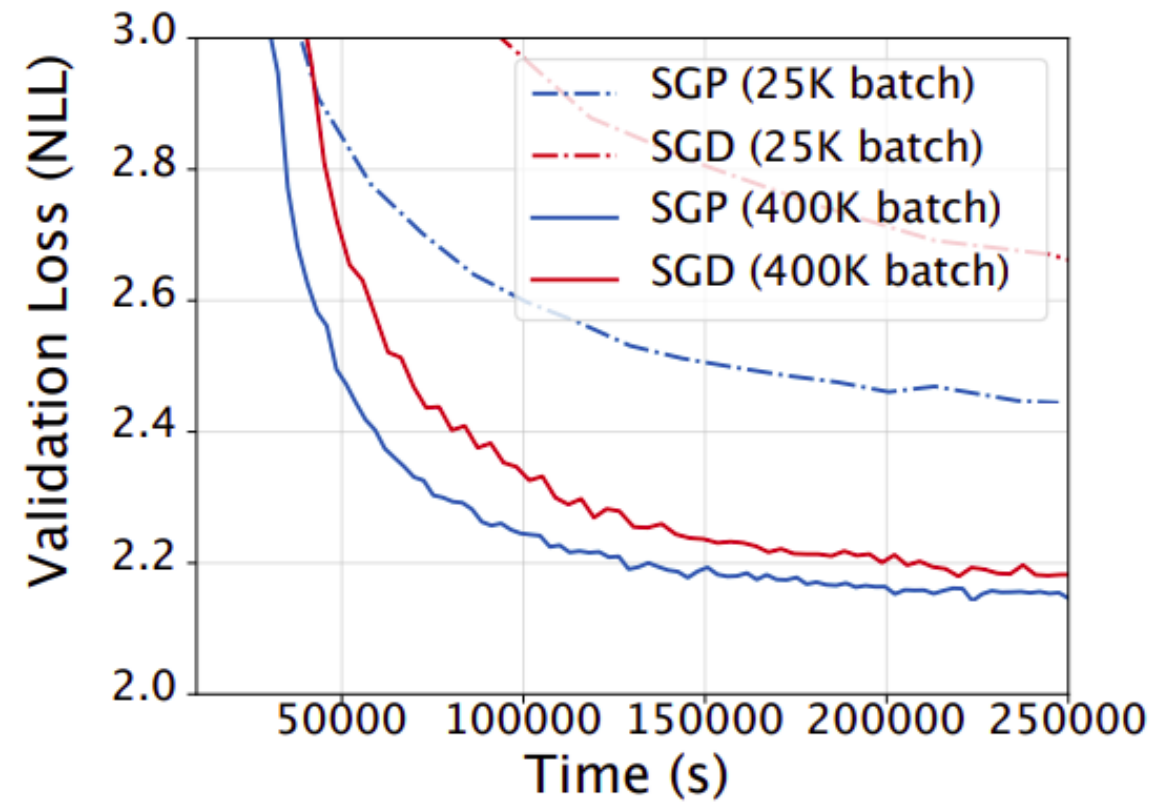
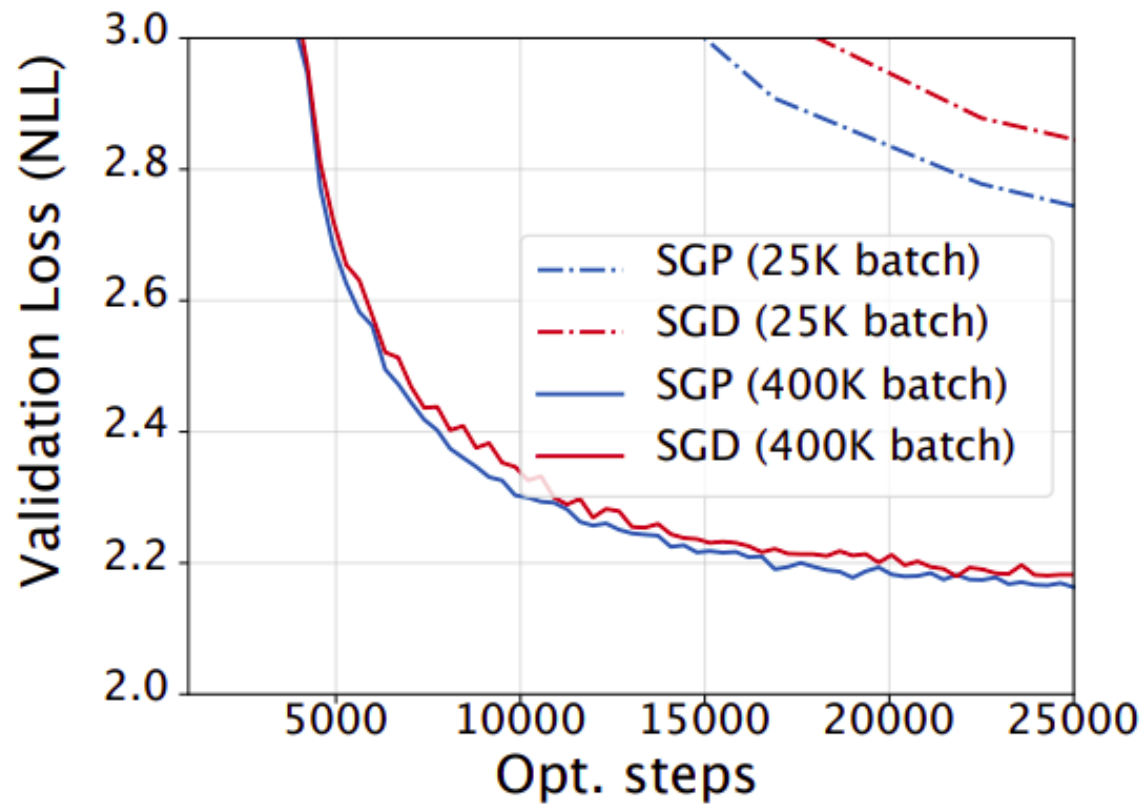
SGP vs ImageNet (ResNet50 + SGD w/ momentum)



Stochastic Gradient Push

Source: <https://arxiv.org/abs/1811.10792>

SGP vs WMT English-German (Transformer, Adam)



Gossip vs All-Reduce

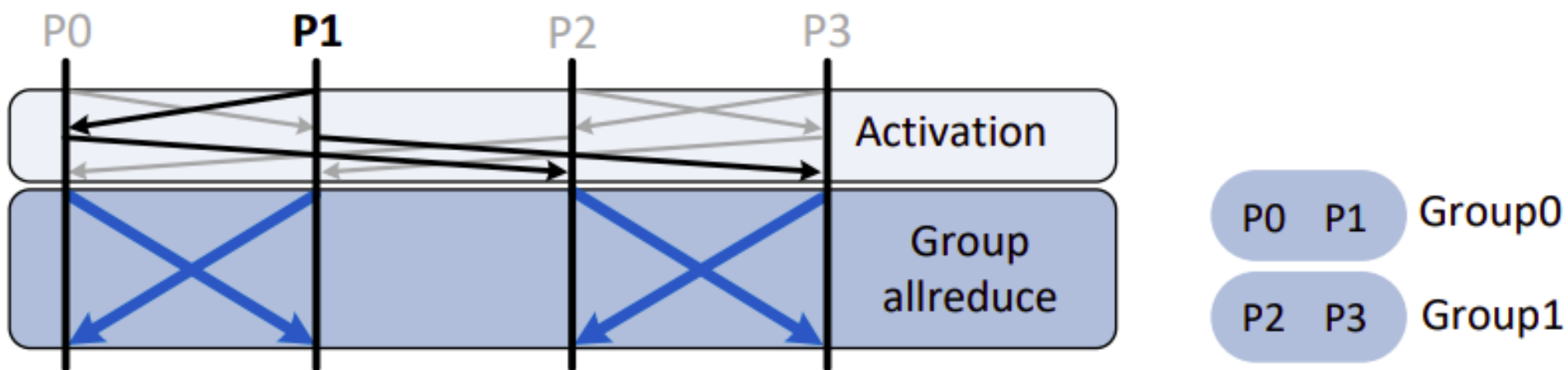
Your thoughts?



Gossip + All-Reduce

Source: arxiv.org/abs/2005.00124

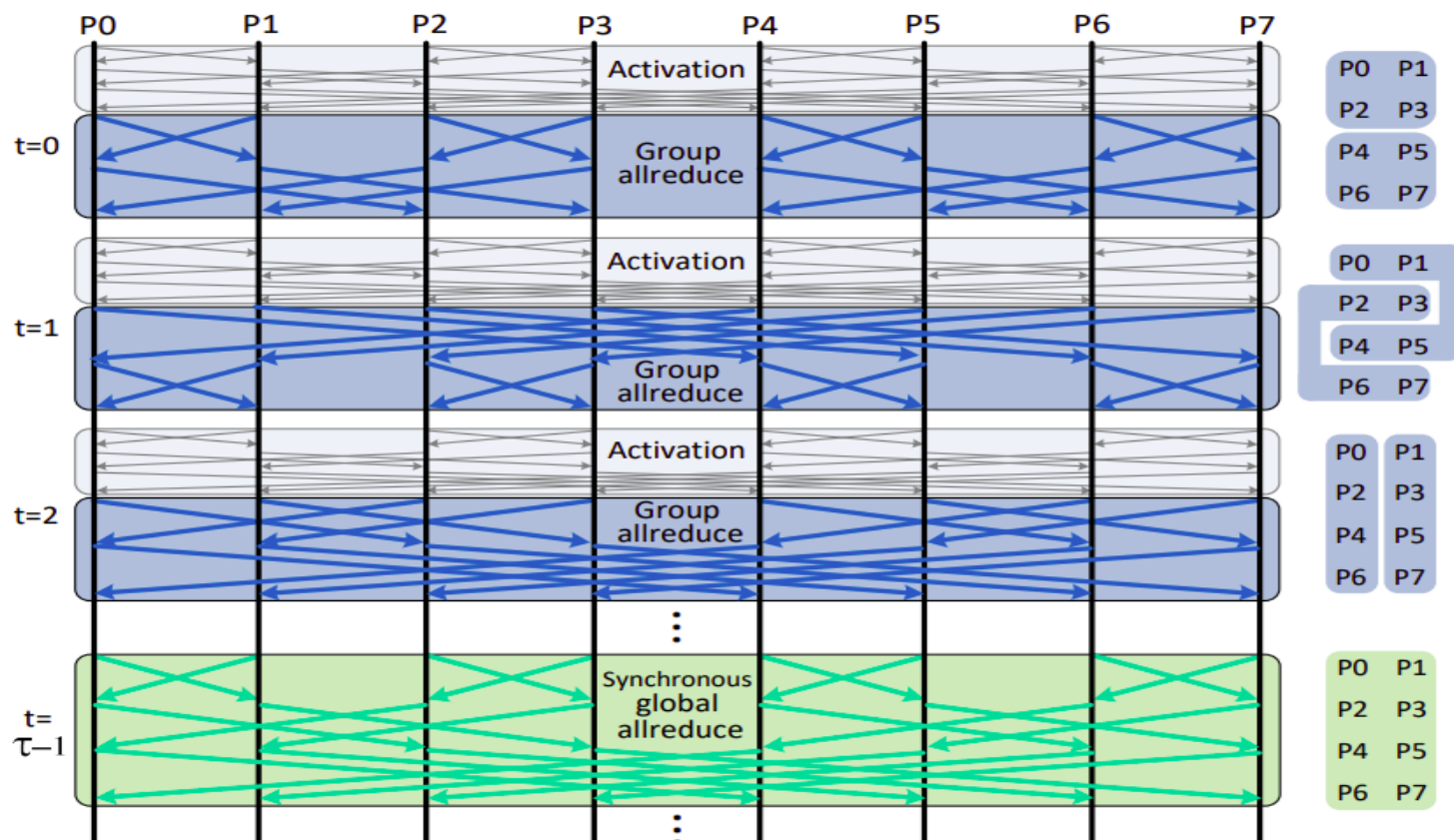
Core idea: run all-reduce in independent groups
You only have to synchronize for your small group
Swap groupmates between iterations



Gossip + All-Reduce

Source: arxiv.org/abs/2005.00124

Core idea: run all-reduce in independent groups
You only have to synchronize for your small group
Swap groupmates between iterations

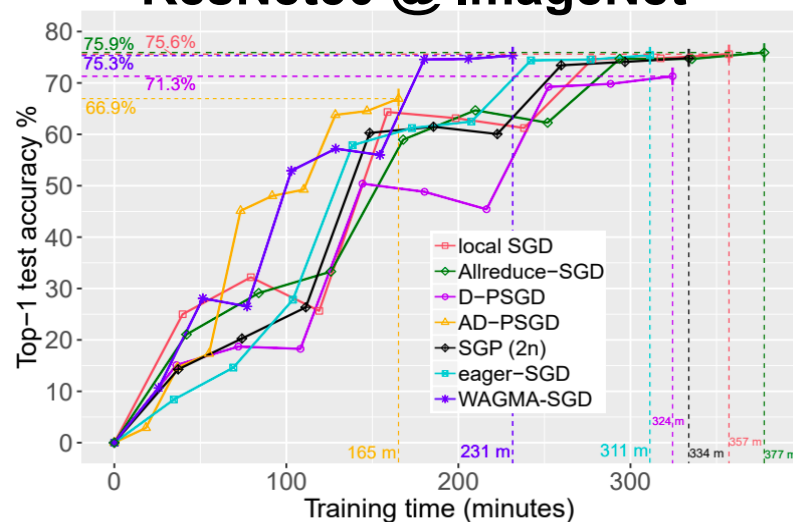


Gossip + All-Reduce

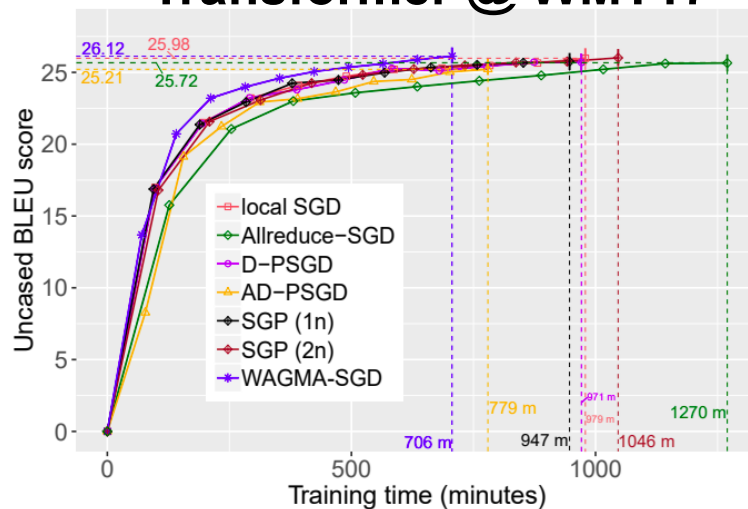
Source: arxiv.org/abs/2005.00124

Experiment setup: up to 1024 GPU,
Natural (or emulated) network latency

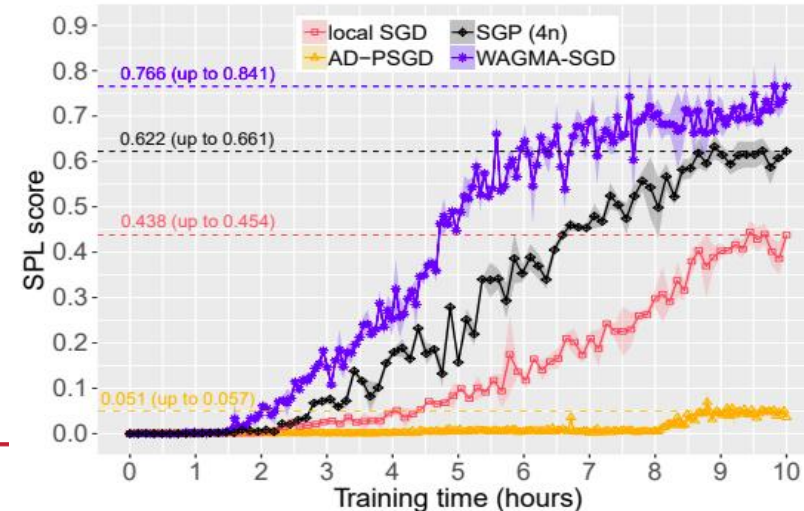
**Image Classification
ResNet50 @ ImageNet**



**Machine Translation
Transformer @ WMT17**



**Reinforcement Learning
DDPO on Habitat**



Data-parallel

- Why?
 - easy to implement
 - can scale to 100s of gpus
 - can be fault-tolerant
 - model must fit in 1 gpu
 - large batches aren't always
 - good for generalization
- 2-4 GPUs & no time – naive data parallel
https://pytorch.org/tutorials/beginner/blitz/data_parallel_tutorial.html
- 4+ GPUs or multiple hosts – distributed (allreduce). github.com/horovod/horovod
- Intro to pytorch distributed: https://pytorch.org/tutorials/intermediate/ddp_tutorial.html or in 15 minutes!
- Somewhat faulty GPU/network: synchronous data parallel + drop stragglers
- Very faulty or uneven resources: asynchronous data parallel (more later)
- Efficient training with large batches: LAMB <https://arxiv.org/abs/1904.00962>
- Dynamically adding or removing resources: <https://github.com/pytorch/elastic>



Q: what if sending tensors during
AllReduce takes too long?



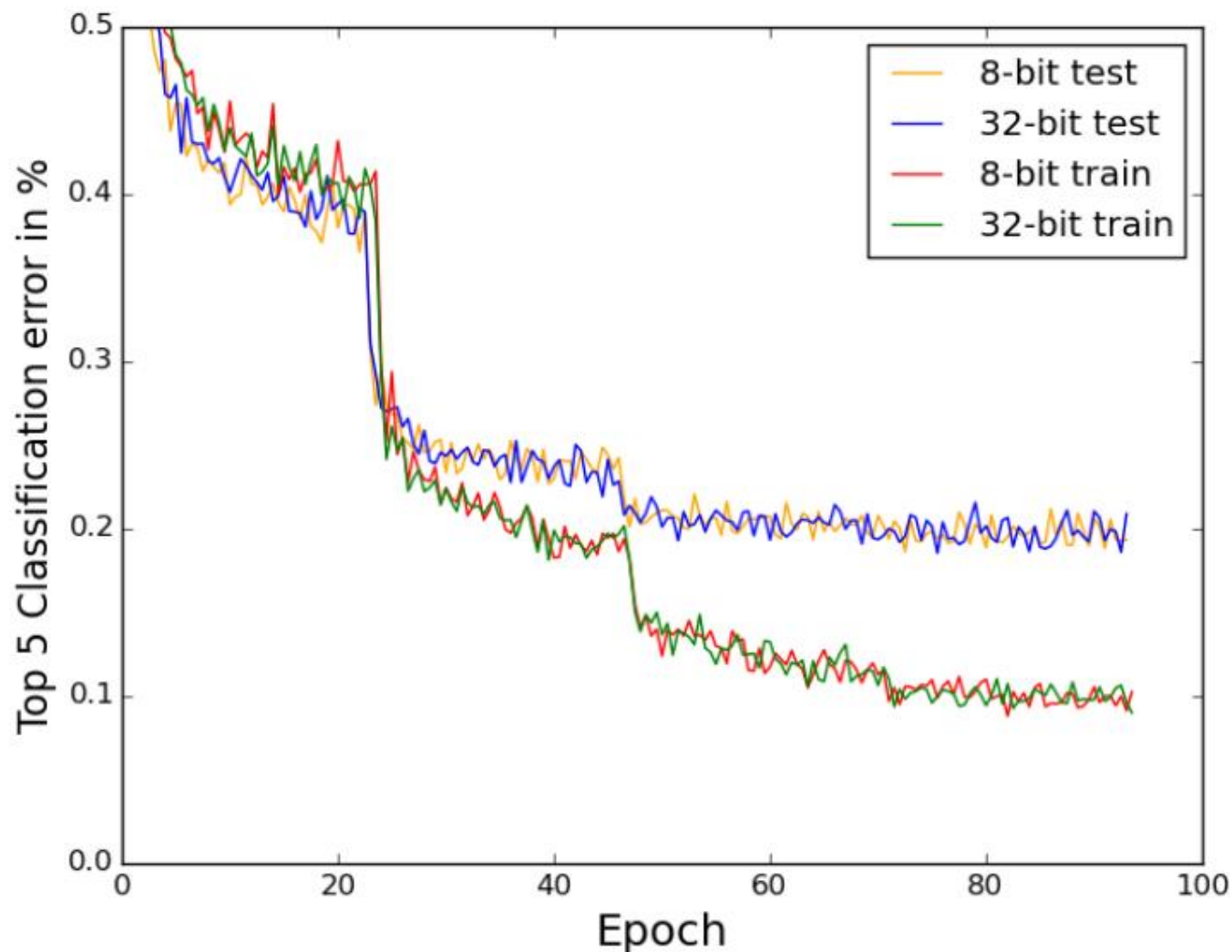
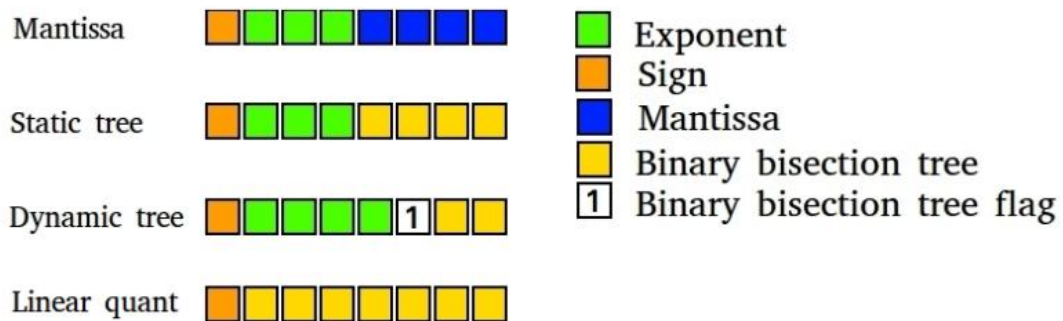
Quantized communication

<https://arxiv.org/abs/1511.04561>

TL;DR

- send data in 8-bit
- all computations in 32-bit
- choose best data format

PROFIT: same quality as float16



Can we compress further?
without losing quality



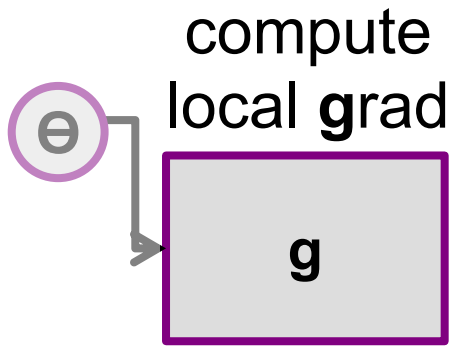
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



time flows left to right

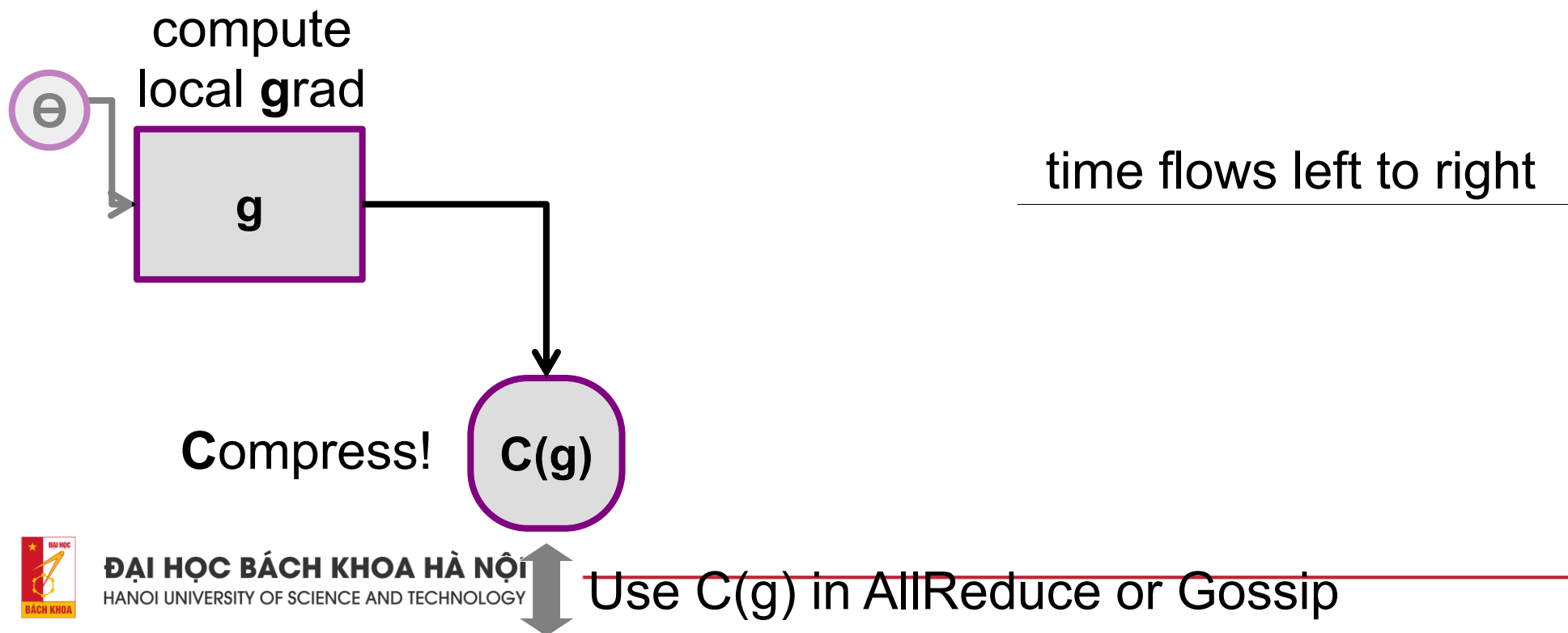
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



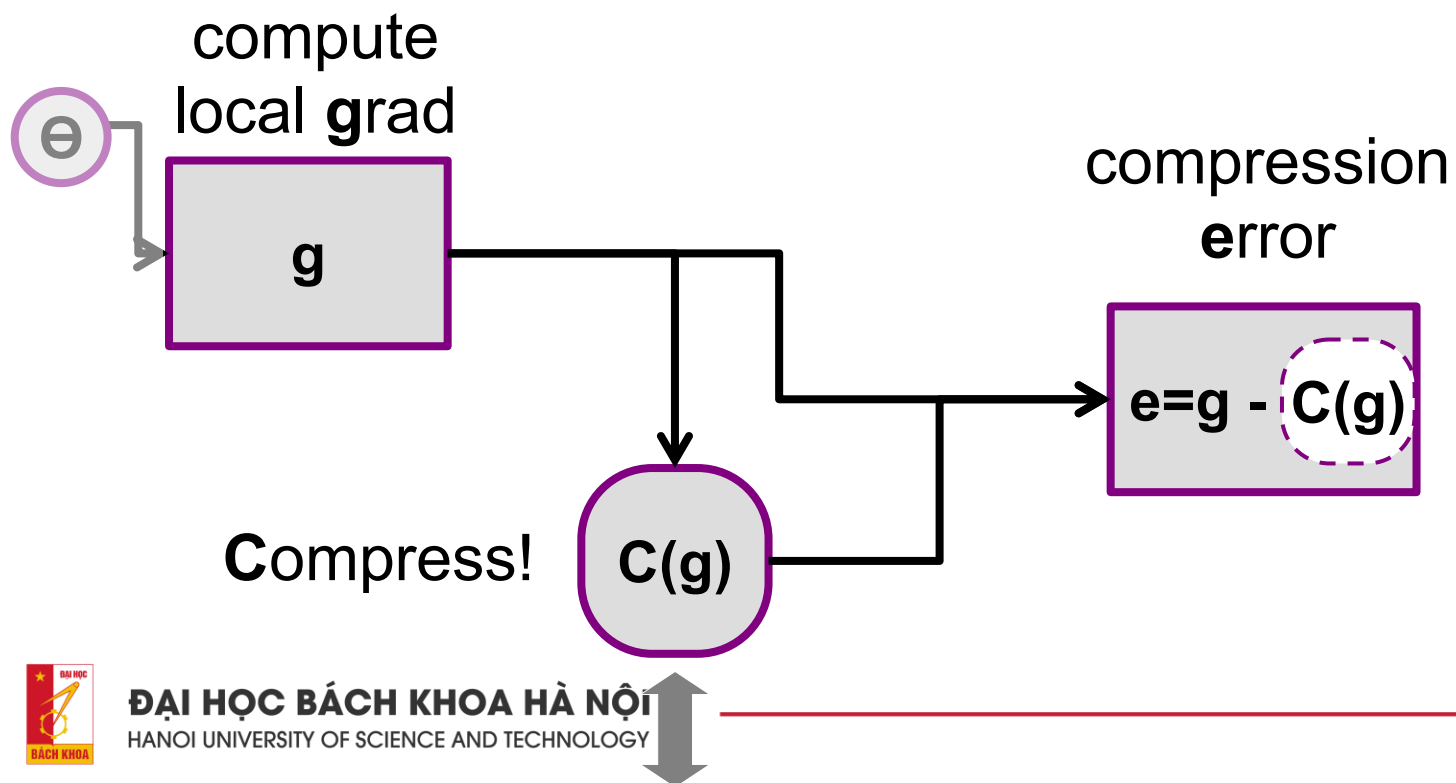
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



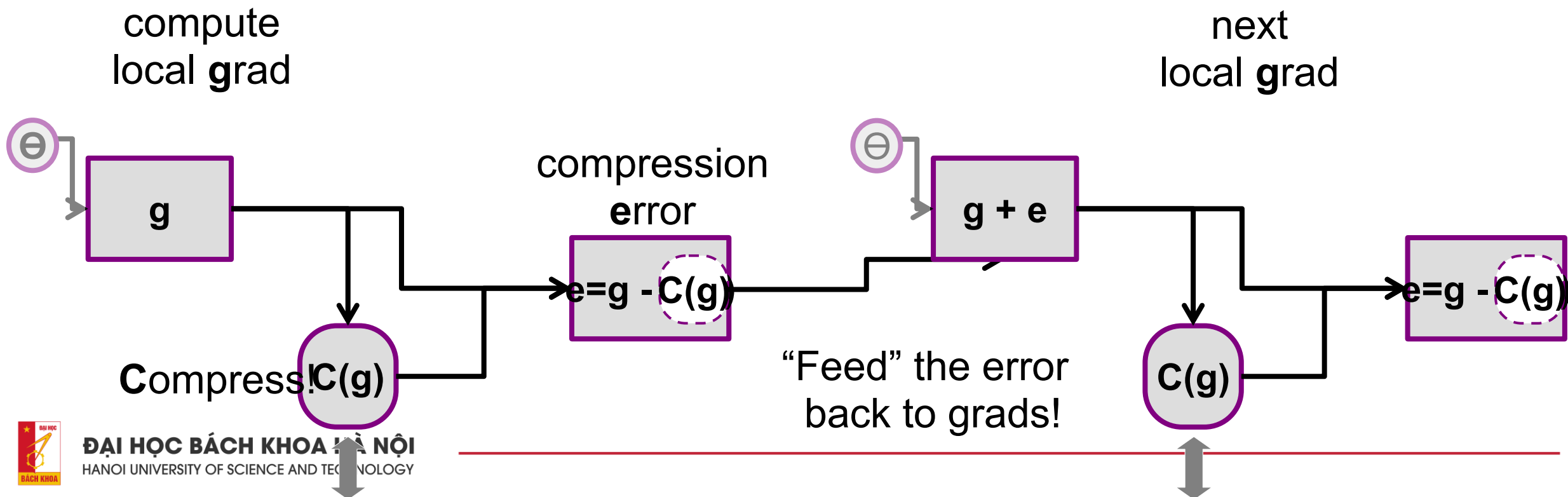
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



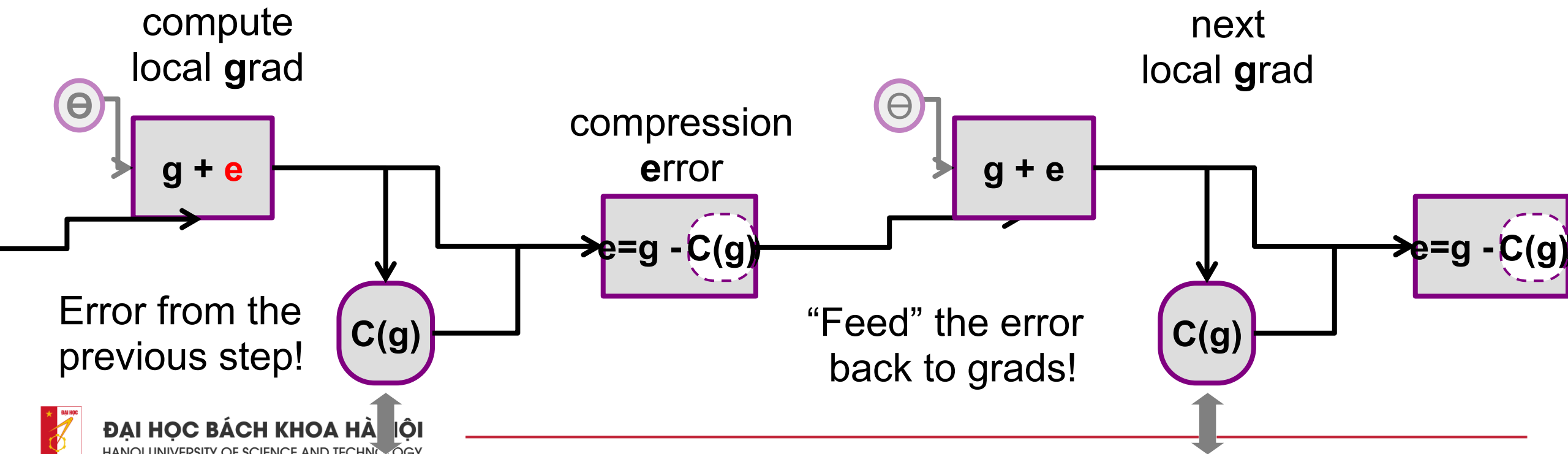
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

```
1: hyperparameters: learning rate  $\gamma$ , momentum parameter  $\lambda$ 
2: initialize model parameters  $\mathbf{x} \in \mathbb{R}^d$ , momentum  $\mathbf{m} \leftarrow \mathbf{0} \in \mathbb{R}^d$ , replicated across workers
3: at each worker  $w = 1, \dots, W$  do
4:   initialize memory  $\mathbf{e}_w \leftarrow \mathbf{0} \in \mathbb{R}^d$ 
5:   for each iterate  $t = 0, \dots$  do
6:     Compute a stochastic gradient  $\mathbf{g}_w \in \mathbb{R}^d$ .
7:      $\Delta_w \leftarrow \mathbf{g}_w + \mathbf{e}_w$  ▷ Incorporate error-feedback into update
8:      $\mathcal{C}(\Delta_w) \leftarrow \text{COMPRESS}(\Delta_w)$ 
9:      $\mathbf{e}_w \leftarrow \Delta_w - \text{DECOMPRESS}(\mathcal{C}(\Delta_w))$  ▷ Memorize local errors
10:     $\mathcal{C}(\Delta) \leftarrow \text{AGGREGATE}(\mathcal{C}(\Delta_1), \dots, \mathcal{C}(\Delta_W))$  ▷ Exchange gradients
11:     $\Delta' \leftarrow \text{DECOMPRESS}(\mathcal{C}(\Delta))$  ▷ Reconstruct an update  $\in \mathbb{R}^d$ 
12:     $\mathbf{m} \leftarrow \lambda \mathbf{m} + \Delta'$ 
13:     $\mathbf{x} \leftarrow \mathbf{x} - \gamma (\Delta' + \mathbf{m})$ 
14:  end for
15: end at
```

PowerSGD: low-rank approx grads + Error Feedback

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

- 1: The update vector Δ_w is treated as a list of tensors corresponding to individual model parameters. Vector-shaped parameters (biases) are aggregated uncompressed. Other parameters are reshaped into matrices. The functions below operate on such matrices independently. For each matrix $M \in \mathbb{R}^{n \times m}$, a corresponding $Q \in \mathbb{R}^{m \times r}$ is initialized from an i.i.d. standard normal distribution.
 - 2: **function** COMPRESS+AGGREGATE(update matrix $M \in \mathbb{R}^{n \times m}$, previous $Q \in \mathbb{R}^{m \times r}$)
 - 3: $P \leftarrow MQ$
 - 4: $P \leftarrow \text{ALL REDUCE MEAN}(P)$ ▷ Now, $P = \frac{1}{W}(M_1 + \dots + M_W)Q$
 - 5: $\hat{P} \leftarrow \text{ORTHOGONALIZE}(P)$ ▷ Orthonormal columns
 - 6: $Q \leftarrow M^\top \hat{P}$
 - 7: $Q \leftarrow \text{ALL REDUCE MEAN}(Q)$ ▷ Now, $Q = \frac{1}{W}(M_1 + \dots + M_W)^\top \hat{P}$
 - 8: **return** the compressed representation $(\hat{P}, Q)$.
 - 9: **end function**
 - 10: **function** DECOMPRESS($\hat{P} \in \mathbb{R}^{n \times r}$, $Q \in \mathbb{R}^{m \times r}$)
 - 11: **return** $\hat{P}Q^\top$
 - 12: **end function**
-

Read More: gradient compression

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/2106.05203> - better EF'21

<https://arxiv.org/abs/1905.13727> - PowerSGD

<https://arxiv.org/abs/2110.03294> - more EF'21

```
1 import torch.distributed.algorithms.ddp_comm_hooks.powerSGD_hook as powerSGD
2
3 ddp_model = nn.parallel.DistributedDataParallel(
4     module=model,
5     device_ids=[rank],
6 )
7
8 state = PowerSGD.PowerSGDState(
9     process_group=process_group,
10    matrix_approximation_rank=1,
11    start_powerSGD_iter=1_000,
12 )
13 ddp_model.register_comm_hook(state, PowerSGD.powerSGD_hook)
```